



UNIVERSITÀ DELLA CALABRIA

DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA,
ELETTRONICA E SISTEMISTICA

Corso di Laurea Magistrale in Ingegneria Informatica

Relazione di progetto per l'insegnamento di

Sistemi Distribuiti e Cloud Computing

Gestore Documenti Smart

Sistema distribuito e containerizzato per la catalogazione,
la classificazione automatica e la ricerca semantica
di documenti amministrativi comunali

Studente: Giuseppe Giulio Misitano

Matricola: 281049

Anno Accademico 2025/2026

Indice

1	Introduzione e Obiettivi	1
1.1	Contesto e dominio applicativo	1
1.2	Obiettivi del progetto	1
1.3	Requisiti funzionali	2
1.4	Requisiti non funzionali	2
1.5	Perimetro del lavoro e vincoli assunti	3
1.6	Struttura della relazione	3
2	Architettura del Sistema	4
2.1	Visione d'insieme	4
2.2	Stile architetturale e compromessi	4
2.3	Scomposizione interna del backend	5
2.4	Separazione fra calcolo e stato	6
2.5	Protocolli di comunicazione	7
2.5.1	Interfaccia REST esposta	8
2.6	Naming, service discovery e trasparenza	10
3	Tecnologie e Strumenti	11
3.1	Panoramica generale	11
3.2	Livello applicativo	11
3.3	Persistenza	12
3.4	Componenti di intelligenza artificiale	12
3.5	Frontend	13
4	Scelte Progettuali e Soluzioni	14
4.1	La pipeline di elaborazione documentale	14
4.2	Gestione dello stato e consistenza	14
4.2.1	Il teorema CAP e la scelta compiuta	14
4.2.2	Consistenza fra due archivi eterogenei	15
4.2.3	Stato derivato materializzato nello schema	16
4.2.4	Idempotenza e riconciliazione	17
4.3	Concorrenza	17
4.3.1	Il modello di esecuzione	17
4.3.2	Conflitti di inizializzazione dello schema del DB	18
4.3.3	Contesa di risorse fra elaborazioni simultanee	18
4.3.4	Gestione thread-safe della cache dei prototipi	19
4.4	Progettazione della ricerca	19
4.4.1	Ricerca full-text	19
4.4.2	Ricerca semantica	20
4.4.3	Ricerca ibrida	20
4.4.4	Ricerca per immagine	21
4.4.5	Documenti simili e suggerimento della tipologia	21

4.5	Tolleranza ai guasti	21
4.5.1	Tassonomia dei guasti e risposte adottate	22
4.5.2	Disponibilità effettiva del servizio e ordine di avvio	22
4.5.3	Strategie di retry e controllo dei timeout	23
4.5.4	Continuità operativa in presenza di errori parziali	23
4.5.5	Limiti noti della tolleranza ai guasti	24
4.6	Scalabilità	24
4.6.1	Scalabilità orizzontale del livello applicativo	24
4.6.2	Colli di bottiglia	25
4.6.3	Ottimizzazioni adottate	25
4.7	Sicurezza	26
5	Deployment e Infrastruttura Cloud	27
5.1	Virtualizzazione a livello di sistema operativo	27
5.2	Costruzione dell'immagine del backend	27
5.3	Costruzione del frontend	28
5.4	Orchestrazione con Docker Compose	28
5.5	Segmentazione della rete	29
5.6	Persistenza dei dati	29
5.7	File per l'ambiente di sviluppo	30
5.8	Configurazione e gestione dei segreti	30
5.9	Avvio del sistema	30
5.10	Estrazione dell'elaborazione in un servizio autonomo	30
5.11	Corrispondenza con i servizi cloud gestiti	31
6	Interfaccia Utente	32
6.1	Tecnologia e deployment	32
6.2	Comunicazione con il backend	32
6.3	Resilienza dell'interfaccia	32
6.4	Le viste principali	34
7	Prompt e Generazioni IA	37
7.1	Prompt formulati e output ottenuti	37

1 Introduzione e Obiettivi

1.1 Contesto e dominio applicativo

Un archivio comunale gestisce documenti eterogenei (delibere, ordinanze, determine, ecc.) il cui reale contenuto informativo risiede spesso in un'immagine scansionata, associata a una struttura di metadati condivisa. Da questa asimmetria derivano due conseguenze principali che guidano il progetto.

In primo luogo, il sistema deve gestire *due nature di dato* radicalmente diverse: I dati descrittivi; ossia le informazioni di base del documento, come il titolo, la data, o l'ufficio di provenienza (metadati: leggeri e fortemente interrogati) e I file veri e propri; ossia le immagini (oggetti binari: grandi e immutabili). Gestirli con la stessa tecnologia è impraticabile, rendendo necessaria una separazione dei livelli di persistenza che costituisce il primo tema distribuito affrontato.

In secondo luogo, la ricerca per parole chiave esatte spesso potrebbe risultare inefficace a causa della variabilità terminologica usata dagli uffici nel corso del tempo (es. *piano regolatore* vs *strumento urbanistico*). Il problema è superato adottando tecniche di rappresentazione vettoriale (*embedding*), che permettono di ricercare per *significato* anziché per stringhe, utilizzando modelli compatti eseguibili in container senza hardware dedicato.

Mettendo insieme le due soluzioni appena citate (ossia l'uso di due archivi separati per dati pesanti e leggeri, e l'uso dell'intelligenza artificiale per cercare i concetti), prende vita il progetto dal nome **Gestore Documenti Smart**. Si tratta di un'applicazione *cloud-native* interamente containerizzata, formata da quattro servizi indipendenti orchestrati tramite Docker Compose e avviabile con un singolo comando.

1.2 Obiettivi del progetto

L'obiettivo generale del progetto è la realizzazione di un sistema distribuito per gestire, catalogare e ricercare documenti tramite tecniche di intelligenza artificiale; utilizzando (come richiesto da traccia) le soluzioni di calcolo, storage e virtualizzazione offerte da Docker Engine.

L'obiettivo generale si articola in quattro traguardi specifici:

1. **Gestione completa dei documenti:** Inserimento (singolo o massivo tramite CSV/JSON) di schede con metadati e immagine di copertina, gestendone l'intero ciclo di vita (creazione, consultazione, modifica e cancellazione).
2. **Opzioni di ricerca avanzata:** Quattro opzioni di interrogazione dell'archivio: testuale (con *stemming* italiano), semantica (tramite *embedding*), ibrida configurabile e per immagine (estraendo automaticamente il testo dalla foto di un documento).
3. **Intelligenza artificiale applicata al dominio.** Riconoscimento ottico dei caratteri, estrazione di parole chiave, suggerimento della tipologia e proposta di documenti affini, il tutto eseguito in locale senza dipendere da servizi esterni.

4. **Architettura distribuita e riproducibile:** Quattro servizi in container su reti segmentate. Lo stato è salvato all'esterno dei container, l'avvio è controllato in modo sicuro e l'applicazione può essere replicata orizzontalmente in caso di necessità.

1.3 Requisiti funzionali

La Tabella 1.1 riporta i requisiti funzionali derivati dalla traccia. La colonna di destra anticipa il capitolo in cui la relativa scelta progettuale viene discussa.

ID	Requisito	Descrizione	Cap.
RF-01	Inserimento singolo	Creazione di un documento compilando un modulo.	6
RF-02	Caricamento massivo	Importazione di più schede da file CSV o JSON, con report dei risultati.	4
RF-03	Catalogazione	Ogni scheda comprende nome, descrizione, tipologia, data, ufficio, firmatari e immagine della prima pagina.	2
RF-04	Ricerca full-text	Ricerca per parole chiave classiche, con ordinamento per rilevanza.	4
RF-05	Ricerca per immagine	Ricerca di un documento partendo da una sua fotografia.	4
RF-06	Documenti simili	Suggerimento automatico di documenti simili.	4
RF-07	Ricerca semantica e ibrida	Ricerca basata sul significato dei termini (semantica) e mista (full-text e semantica).	4
RF-08	Operazioni CRUD	Lettura, aggiornamento e cancellazione dei documenti.	4
RF-09	Virtualizzazione	Sistema interamente containerizzato e orchestrato, avviabile con un unico comando.	5
RF-10	Estrazione di parole chiave	Individuazione automatica dei termini più rappresentativi dal testo riconosciuto.	4
RF-11	Suggerimento della tipologia	Suggerimento automatico della categoria amministrativa del documento.	4

Tabella 1.1: Requisiti funzionali del sistema e capitolo di riferimento.

1.4 Requisiti non funzionali

I requisiti non funzionali, presenti nella Tabella 1.2, definiscono l'architettura del sistema e le sue garanzie. Di seguito quelli fissati per questo progetto, che verranno discussi nel Capitolo 4.

Il primo requisito non funzionale che vedremo in tabella è il più importante: in un archivio amministrativo i dati non possono rischiare di essere disallineati, nemmeno temporaneamente. Per questo il sistema privilegia la consistenza rispetto alla disponibilità, come discusso in §4.2.1.

ID	Attributo	Requisito e criterio di accettazione
RNF-01	Consistenza	Metadati e dati vettoriali sempre allineati (consistenza forte).
RNF-02	Disponibilità	Se un componente si guasta, il resto del sistema continua a funzionare.
RNF-03	Latenza	Ricerche molto veloci; le operazioni pesanti vengono gestite in background.
RNF-04	Scalabilità	L'applicazione può essere replicata facilmente senza modificare il codice.
RNF-05	Portabilità	Il sistema è pronto per essere spostato sul cloud senza modifiche al codice.
RNF-06	Sicurezza	Database isolati, nessun privilegio di amministratore e password protette.
RNF-07	Riproducibilità	Installazioni sempre identiche bloccando le versioni dei software utilizzati.

Tabella 1.2: Requisiti non funzionali e criteri di accettazione.

1.5 Perimetro del lavoro e vincoli assunti

Per valutare correttamente le scelte progettuali, è necessario chiarire i limiti e i vincoli del progetto.

Vincolo tecnologico: Come richiesto, è stato usato esclusivamente **Docker Engine**, senza ricorrere a servizi cloud o a sistemi di gestione dei container più complessi. Sono stati scelti componenti standard (come API S3 e PostgreSQL) per rendere il sistema facilmente trasferibile in futuro sul cloud cambiando solo le configurazioni, senza dover riscrivere il codice (§5.11).

Vincolo infrastrutturale: Il sistema è stato sviluppato e testato su un unico computer fisico. Replicare i dati sulla stessa macchina creerebbe un falso senso di sicurezza (se si guasta il computer, si perde tutto). Per questo si è evitato di farlo, specificando però nel documento come il sistema andrebbe adattato per funzionare su più macchine.

Fuori perimetro: Poiché non richiesto dalla traccia, non è stata inserita l'autenticazione (tutte le operazioni sono anonime) e la relativa gestione degli accessi (§??). Non sono stati realizzati neanche sistemi automatici per il rilascio del software (CI/CD) o la gestione dell'infrastruttura come codice, che vengono però discussi tra i possibili sviluppi futuri (§??).

1.6 Struttura della relazione

Il documento procede a livelli di dettaglio crescenti, illustrando e motivando in ogni capitolo le soluzioni adottate rispetto alle alternative scartate. Il Capitolo 2 presenta la visione d'insieme del sistema, lo stile architetturale e i protocolli di comunicazione, mentre il Capitolo 3 analizza le tecnologie scelte. Il Capitolo 4 rappresenta il cuore del documento, affrontando i temi della consistenza dei dati, della concorrenza, della tolleranza ai guasti e della scalabilità. Successivamente, il Capitolo 5 descrive la gestione dei container e la predisposizione per il cloud; il Capitolo 6 tratta l'interfaccia utente e la gestione degli errori o dei carichi temporanei. Infine, il Capitolo ?? sintetizza i risultati e gli sviluppi futuri del progetto, mentre il Capitolo 7 documenta l'utilizzo dell'intelligenza artificiale durante lo sviluppo.

2 Architettura del Sistema

2.1 Visione d'insieme

Il sistema è composto da quattro servizi containerizzati, ciascuno dei quali indipendente e con un compito specifico. Questi comunicano attraverso due reti virtuali separate e utilizzano tre volumi per il salvataggio permanente dei dati. La struttura complessiva è illustrata nella Figura 2.1.

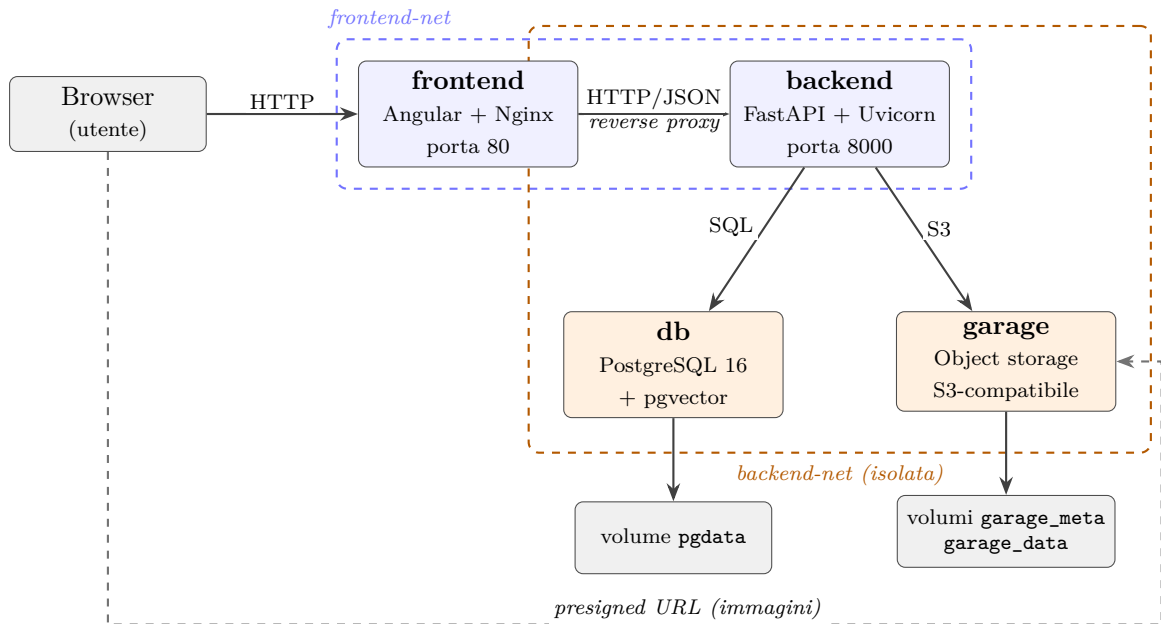


Figura 2.1: Architettura complessiva: quattro servizi, due reti virtuali segmentate e tre volumi persistenti. Il servizio **backend** è l'unico collegato a entrambe le reti; **db** e **garage** non sono raggiungibili dalla rete del frontend.

Il browser comunica esclusivamente con il servizio **frontend**: le risorse statiche sono fornite direttamente da Nginx, mentre le richieste che iniziano con `/api/` vengono inoltrate al backend. Per elaborarle, il backend recupera i metadati e i dati vettoriali da PostgreSQL, e i file da Garage. Le immagini rappresentano l'unica, voluta eccezione: per evitare di farle passare dal backend, il sistema genera un link temporaneo. Il browser usa questo link per scaricare l'immagine *direttamente* dall'archivio (la freccia tratteggiata in figura). In questo modo, il backend risparmia memoria e banda, evitando di sprecare risorse per trasferire file di grandi dimensioni (§??).

2.2 Stile architetturale e compromessi

Il sistema adotta un'architettura **a tre livelli (three-tier) containerizzata**, in cui il livello applicativo è strutturato come un **monolite modulare** e non come un insieme di microservizi. Questa è la scelta progettuale più importante e merita di essere spiegata confrontandola con l'alternativa scartata.

L’alternativa a microservizi: La natura del progetto suggerirebbe di dividere il sistema in almeno tre microservizi: gestione documentale (salvataggio e metadati), elaborazione (OCR e parole chiave) e ricerca (motore semantico). Questa divisione avrebbe senso perché i tre moduli usano le risorse in modo diverso: il primo dipende molto dalla velocità di lettura/scrittura (I/O), mentre gli altri due richiedono molta potenza di calcolo (CPU).

Perché è stata scartata: Nonostante i potenziali vantaggi, dividere il sistema avrebbe introdotto complicazioni ingiustificate per le dimensioni del progetto:

- *Consistenza dei dati:* Attualmente, il salvataggio dei metadati e la generazione dei dati vettoriali avvengono simultaneamente in un’unica transazione sicura. Se i servizi fossero separati, si perderebbe questa garanzia nativa e bisognerebbe ricostruirla da zero sviluppando meccanismi di sincronizzazione complessi.
- *Consumo di memoria:* Oggi i tre moduli condividono un’unica copia in memoria del modello di intelligenza artificiale (circa 100 MB). Separandoli in microservizi diversi, ciascuno dovrebbe caricare la propria copia, spreco di RAM.
- *Costo operativo:* Gestire tre servizi separati significherebbe costruire, monitorare e far dialogare tre componenti diversi, a fronte di un carico di lavoro che un singolo blocco gestisce già senza problemi.

Il compromesso accettato e la sua reversibilità: Accettare che l’applicazione sia un “blocco unico” comporta dei limiti evidenti: un errore in una singola funzione può bloccare l’intero programma, e se un solo componente ha bisogno di più potenza, bisogna per forza potenziare tutto il blocco. Questa scelta è stata però progettata per essere *reversibile*. Il codice è organizzato a “scompartimenti stagni”: ogni strumento (come l’OCR o la ricerca semantica) è strettamente isolato nel proprio modulo (`ocr_service`, `embedding_service`, `keyword_service`, `garage_service`). Se in futuro si vorrà staccare uno di questi moduli per farlo diventare un servizio autonomo, basterà solo cambiare il modo in cui “parla” con il resto del programma (passando da un semplice comando interno a una chiamata di rete), senza dover stravolgere il codice (§5.10).

Va infine precisato una distinzione importante: la scelta del blocco unico (“monolite”) riguarda *esclusivamente* il livello applicativo. Il salvataggio dei dati è già distribuito su due servizi indipendenti, con regole e funzionamenti propri. Di conseguenza, il sistema nel suo complesso è a tutti gli effetti distribuito: deve gestire comunicazioni di rete, guasti parziali, avvii sincronizzati e coerenza tra archivi eterogenei, a prescindere da come è stato impacchettato il codice dell’applicazione.

2.3 Scomposizione interna del backend

All’interno del container applicativo (il backend) il codice è organizzato in quattro livelli sovrapposti, ciascuno dei quali può rivolgersi soltanto a quello immediatamente sottostante. La Tabella 2.1 ne riassume ruoli e responsabilità.

Livello	Cartella	Responsabilità
Presentazione	<code>app/routers/</code>	Riceve le richieste web (HTTP), controlla i parametri e restituisce le risposte. Non prende decisioni sul funzionamento del sistema.
Contratto	<code>app/schemas/</code>	Definisce la forma che devono avere i dati in entrata e in uscita, separando ciò che vede l'utente da come i dati sono salvati internamente.
Servizio	<code>app/services/</code>	Contiene il vero “cervello” dell'applicazione: le regole di funzionamento e i collegamenti con gli strumenti esterni.
Persistenza	<code>app/models/</code> , <code>app/database.py</code>	Si occupa unicamente di salvare e leggere i dati dal database, definendo tabelle, indici e regole di archiviazione.

Tabella 2.1: Livelli interni del backend e relative responsabilità.

Mantenere separato il livello di “contratto” (i dati scambiati con l'esterno) da quello di “persistenza” (i dati salvati nel database) è fondamentale per poter aggiornare il sistema senza causare disservizi agli utenti.

Ad esempio:

- il modulo per creare un documento (`DocumentoCreate`) chiede all'operatore di compilare solo alcuni campi;
- la risposta del sistema (`DocumentoResponse`) ne aggiunge altri generati in automatico;
- il modulo per le modifiche (`DocumentoUpdate`) li rende facoltativi per permettere modifiche parziali.

Nessuno di questi moduli corrisponde esattamente a come il documento viene salvato per intero nel database, che include dati tecnici (come i vettori a 384 dimensioni) inutili per l'utente e quindi sempre nascosti.

Questa separazione offre anche un importante vantaggio in termini di sicurezza: poiché le regole per l'importazione massiva derivano direttamente da quelle di creazione base, la modifica di un singolo vincolo si applica automaticamente a entrambi i flussi, evitando qualsiasi disallineamento o inserimento non previsto.

2.4 Separazione fra calcolo e stato

Il principio fondamentale che guida la distribuzione dei compiti tra i quattro servizi è la separazione tra chi elabora i dati e chi ne gestisce la persistenza.

I due servizi applicativi, backend e frontend, sono progettati per essere stateless: non scrivono dati persistenti sul proprio filesystem, non mantengono sessioni in memoria e non dipendono da informazioni conservate localmente tra una richiesta e l'altra.

Lo stato persistente viene quindi esternalizzato dai container applicativi e affidato ai componenti dedicati alla sua gestione. PostgreSQL conserva i dati strutturati e gli indici, mentre Garage gestisce le immagini delle scansioni. I modelli di AI, invece, fanno parte dell'immagine del container e sono quindi considerati parte dell'ambiente di esecuzione.

La Tabella 2.2 riassume la distribuzione delle diverse categorie di dati tra i componenti del sistema, indicando per ciascuna il responsabile della memorizzazione e il motivo della scelta.

Categoria	Gestione	Motivo della scelta
Metadati strutturati	PostgreSQL	Supporta interrogazioni relazionali e garantisce integrità e transazioni.
Vettori di embedding	PostgreSQL (<i>pgvector</i>)	Restano coerenti con i metadati a cui si riferiscono, grazie alla stessa transazione.
Indice full-text	PostgreSQL (colonna tsvector)	È derivato dai metadati e viene aggiornato automaticamente dal database tramite <i>trigger</i> .
Immagini delle scansioni	Garage	È adatto a file di grandi dimensioni, immutabili e gestiti come oggetti completi.
Modelli di AI	Immagine del container	Sono inclusi nell'immagine e quindi fanno parte dell'ambiente di esecuzione, non dello stato persistente.
Configurazione e segreti	Variabili d'ambiente	Sono forniti dall'ambiente e possono variare senza modificare l'immagine del container (§5.8).

Tabella 2.2: Gestione dei dati e degli elementi dell'ambiente di esecuzione.

Questa caratteristica rende possibile aumentare il numero di istanze del backend in base al carico di lavoro. Poiché le istanze non conservano stato locale, sono intercambiabili: una richiesta può essere gestita da qualsiasi istanza senza differenze. Questa possibilità sarà verificata in §4.6.1.

Un'eccezione riguarda i compiti eseguiti in background, che rimangono nel processo che li ha avviati. Non sono vero e proprio *stato*, perché nessun dato dipende esclusivamente da essi, ma rappresentano lavoro in corso che non viene conservato. Se il container viene arrestato, il lavoro in corso può quindi andare perso. Si tratta di un limite dell'implementazione, analizzato in §4.5.5.

2.5 Protocolli di comunicazione

Il sistema utilizza tre protocolli principali [2], scelti in base alla natura dei diversi collegamenti.

La Tabella 2.3 riassume i protocolli utilizzati e il motivo della loro scelta.

Collegamento	Protocollo	Motivo della scelta
Browser → frontend	HTTP/1.1	È universalmente supportato dai browser senza componenti aggiuntivi.
Frontend → backend	HTTP/JSON (REST)	Offre un'interfaccia semplice, leggibile e ispezionabile, senza generazione di codice o artefatti condivisi.
Backend → database	Protocollo PostgreSQL su TCP	Utilizza il protocollo nativo PostgreSQL e un pool di connessioni riutilizzabili.
Backend → storage	API S3 su HTTP	È lo standard <i>de facto</i> per l'object storage e permette di sostituire l'implementazione senza modificare il codice (§5.11).
Browser → storage	HTTP con firma SigV4	Permette al browser di accedere direttamente alle immagini tramite un URL temporaneo firmato dal backend.

Tabella 2.3: Protocolli utilizzati nel sistema e motivazione della scelta.

REST anziché gRPC: Per la comunicazione tra frontend e backend è stato valutato anche gRPC, che offre una serializzazione binaria più compatta e un contratto tipizzato verificato in compilazione. È stato però preferito REST per tre motivi:

1. I browser non supportano gRPC nativamente, quindi sarebbe necessario aggiungere un componente intermedio.
2. Il volume di dati scambiato è ridotto e le immagini, che rappresentano la parte più pesante, seguono un percorso separato.
3. HTTP/JSON è facilmente ispezionabile e permette di riprodurre le chiamate direttamente da terminale, semplificando la diagnosi dei problemi.

Comunicazione sincrona anziché tramite broker di messaggi: Il sistema non utilizza un broker di messaggi (componente che riceve, conserva e distribuisce messaggi tra servizi permettendo loro di comunicare in modo asincrono e disaccoppiato).

Per semplicità, il backend comunica direttamente e in modo sincrono con i servizi di persistenza. Un broker, come RabbitMQ o SQS, sarebbe particolarmente utile per rendere più robusta l'elaborazione OCR, conservando i compiti anche in caso di arresto del servizio che li ha generati. Tuttavia, introdurrebbe un ulteriore servizio da gestire e una maggiore complessità architetturale, a fronte di un beneficio limitato al contesto del progetto.

L'asincronia necessaria è stata quindi ottenuta tramite i *background task* di FastAPI (§4.1), accettando il limite della possibile perdita dei compiti in corso, discusso in §4.5.5.

2.5.1 Interfaccia REST esposta

Le rotte sono organizzate in base alla risorsa e suddivise in tre *router*. Per rendere più chiara l'interfaccia, gli endpoint sono presentati separatamente in base alla loro funzione.

REST dei documenti: Questo gruppo raccoglie le operazioni per creare, modificare, eliminare e consultare i documenti, oltre alle funzionalità di gestione degli embedding.

Metodo	Percorso	Funzione
GET	/health	Verifica che il servizio sia raggiungibile e che il database sia disponibile.
POST	/api/documenti	Crea una scheda e genera il relativo embedding.
GET	/api/documenti	Restituisce l'elenco paginato dei documenti.
GET	/api/documenti/statistiche	Restituisce i conteggi per stato di elaborazione.
GET	/api/documenti/{id}	Restituisce il dettaglio di una scheda.
PATCH	/api/documenti/{id}	Modifica una scheda e, se necessario, aggiorna l'embedding.
DELETE	/api/documenti/{id}	Elimina la scheda e l'immagine associata.
GET	/api/documenti/{id}/simili	Restituisce i documenti semanticamente più simili.
POST	/api/documenti/suggerisci-tipologia	Restituisce le tipologie più probabili, ordinate per confidenza.
POST	.../manutenzione/ricostruisci-embedding	Ricostruisce gli embedding mancanti (§4.2.4).

Tabella 2.4: Endpoint REST per la gestione dei documenti.

REST per upload e immagini: Questi endpoint gestiscono il caricamento delle immagini e l'importazione massiva dei documenti.

Metodo	Percorso	Funzione
POST	/api/upload/immagine/{id}	Carica un'immagine e avvia l'elaborazione in background.
GET	/api/upload/immagine/{id}/url	Genera un URL firmato per leggere direttamente l'immagine.
POST	/api/upload/massivo	Importa dati da CSV o JSON e restituisce un rapporto sull'operazione.

Tabella 2.5: Endpoint REST per upload e immagini.

REST per la ricerca: Questo gruppo espone le diverse modalità di ricerca disponibili: full-text, semantica, ibrida e tramite immagine.

Metodo	Percorso	Funzione
GET	/api/ricerca	Esegue una ricerca full-text.
GET	/api/ricerca/semantica	Esegue una ricerca basata sul significato dei vettori.
GET	/api/ricerca/ibrida	Combina la ricerca full-text e quella semantica.
POST	/api/ricerca/immagine	Esegue l'OCR sull'immagine e una ricerca ibrida sul testo estratto.

Tabella 2.6: Endpoint REST per la ricerca.

Due dettagli di progettazione meritano una nota.

Il primo riguarda l'uso di **PATCH** anziché **PUT**: PUT sostituisce l'intera risorsa, quindi per modificare il solo campo ufficio sarebbe necessario reinviare tutta la scheda. Questo

potrebbe sovrascrivere dati aggiornati nel frattempo dall'elaborazione in background, come il testo riconosciuto e le parole chiave. PATCH, invece, permette di modificare solo i campi interessati, evitando questo problema.

Il secondo riguarda l'**ordine di dichiarazione delle rotte**. FastAPI valuta le rotte nell'ordine in cui sono registrate: `/api/documenti/statistiche` deve quindi precedere `/api/documenti/{id}`. In caso contrario, `statistiche` verrebbe interpretato come valore di `id` e la richiesta fallirebbe durante la validazione.

2.6 Naming, service discovery e trasparenza

In un sistema distribuito, il *naming* riguarda il modo in cui un nodo individua l'indirizzo di un altro prima di comunicare con esso. Il progetto risolve questo problema senza introdurre un registro dei servizi, utilizzando il DNS interno che Docker Engine attiva sulle reti definite dall'utente. Il nome di ciascun servizio dichiarato nel file di orchestrazione è infatti automaticamente risolvibile dagli altri container della stessa rete.

Nel codice e nella configurazione non compaiono quindi indirizzi IP: il backend utilizza `db:5432` e `garage:3900`, mentre Nginx inoltra a `backend:8000`. Poiché gli indirizzi dei container sono assegnati dinamicamente e possono cambiare a ogni ricreazione, riferirsi direttamente agli IP renderebbe la configurazione fragile. Il DNS interno mantiene invece stabili i riferimenti ai servizi.

Questo meccanismo permette anche di replicare un servizio. Quando sono presenti più istanze con lo stesso nome, il DNS restituisce gli indirizzi delle diverse istanze in ordine variabile. Le richieste possono così essere distribuite tra i container disponibili secondo un comportamento *round-robin*, senza configurazione aggiuntiva (§4.6.1).

La Tabella 2.7 riassume le principali forme di trasparenza presenti nel sistema, cioè i casi in cui i dettagli della distribuzione dei componenti rimangono nascosti al client, indicando anche quelle realizzate in parte.

Forma	Stato	Realizzazione nel sistema
Accesso	✓	Il client utilizza una sola origine HTTP, senza distinguere se la richiesta è gestita da Nginx o inoltrata a FastAPI.
Locazione	✓	I servizi sono raggiunti tramite nomi logici e non tramite indirizzi IP, grazie al DNS interno.
Migrazione	✓	Un container può essere ricreato con un indirizzo diverso senza modificare la configurazione degli altri servizi.
Replicazione	Parziale	Le repliche del backend sono trasparenti al client, mentre i servizi di persistenza non sono replicati per la ragione infrastrutturale descritta in §1.5.
Concorrenza	✓	Le transazioni PostgreSQL e il <i>lock</i> consultivo usato per l'inizializzazione dello schema (§4.3.2) nascondono al client la gestione delle richieste concorrenti.
Guasto	Parziale	Le sonde di prontezza e le politiche di riavvio gestiscono i guasti transitori all'avvio, mentre un guasto del database durante l'esecuzione viene restituito al client come errore.

Tabella 2.7: Forme di trasparenza realizzate dal sistema.

3 Tecnologie e Strumenti

In questo capitolo vengono elencate le tecnologie impiegate giustificandone la scelta. Il criterio adottato è stato il seguente: a parità di funzionalità è stata preferita la soluzione che espone un'interfaccia standard, così che il componente locale possa essere sostituito dal corrispettivo servizio gestito senza modifiche al codice applicativo.

3.1 Panoramica generale

Livello	Tecnologia	Versione	Ruolo
Virtualizzazione	Docker Engine	—	Isolamento dei processi e delle risorse
Orchestrazione	Docker Compose	v2	Definizione dichiarativa dello stack
Backend	Python	3.11	Linguaggio del livello applicativo
	FastAPI	0.115.12	Framework web asincrono
	Uvicorn	0.34.3	Server ASGI
	SQLAlchemy	2.0.41	Mappatura oggetto-relazionale
	Pydantic	v2	Validazione dei contratti
Persistenza	PostgreSQL	16	Metadati, indice full-text, vettori
	pgvector	0.4.2	Tipo vettoriale e indice HNSW
	Garage	2.3.0	Object storage S3-compatibile
	boto3	1.39.10	Client S3
AI / NLP	sentence-transformers	5.0.0	Generazione degli embedding
	all-MiniLM-L6-v2	—	Modello a 384 dimensioni
	Tesseract	5.x	Motore OCR
	KeyBERT	0.8.5	Estrazione di parole chiave
	spaCy	3.8.7	Risorse linguistiche italiane
Frontend	Angular	19.2	Applicazione a pagina singola
	TypeScript	5.7	Linguaggio del frontend
	Nginx	alpine	Server statico e reverse proxy

Tabella 3.1: Tecnologie impiegate e relative versioni.

3.2 Livello applicativo

Python e FastAPI: La scelta di Python è legata soprattutto alla disponibilità di librerie mature per OCR, *embedding* ed estrazione di parole chiave, che in altri ambienti richiederebbero processi esterni o servizi remoti. FastAPI è stato preferito a Flask e Django per tre caratteristiche utilizzate concretamente nel progetto: il modello ASGI, che permette di gestire gli endpoint nel ciclo di eventi o tramite un pool di thread, come discusso in §4.3.1; l'integrazione con Pydantic, che automatizza la validazione dei dati a partire dai tipi dichiarati; infine, la generazione automatica della specifica OpenAPI e della documentazione interattiva, utilizzata anche per verificare e diagnosticare il funzionamento dell'API.

SQLAlchemy: L'accesso ai dati è gestito tramite un *Object-Relational Mapper* (ORM), che permette di lavorare con il database relazionale attraverso gli oggetti del linguaggio. In questo ruolo è simile a Hibernate nell'ecosistema Java rispetto alla specifica JPA. È stata scelta la **versione 2.0 di SQLAlchemy** che supporta la tipizzazione statica e l'integrazione con

pgvector, un'estensione di PostgreSQL che permette di memorizzare e confrontare vettori, utilizzati nel progetto per gli *embedding*. Questa integrazione consente di definire direttamente nel modello anche l'**indice HNSW**, una struttura che rende più efficiente la ricerca dei vettori simili. In questo modo, la struttura del database rimane descritta in un unico punto, senza dover ricorrere a uno script SQL separato.

3.3 Persistenza

PostgreSQL con pgvector, anziché un database vettoriale dedicato: La ricerca semantica avrebbe potuto essere affidata a un database vettoriale specializzato, come Pinecone, Weaviate o Qdrant, più adatto a gestire collezioni di grandi dimensioni. È stato invece scelto pgvector, un'estensione di PostgreSQL che permette di memorizzare vettori e calcolarne la distanza. Il vantaggio principale è che gli *embedding* vengono memorizzati nella stessa riga dei metadati del documento. In questo modo, documento e relativo vettore vengono creati, modificati ed eliminati nella stessa transazione, garantendo che rimangano sempre sincronizzati.

Con un database vettoriale separato, questa sincronizzazione dovrebbe essere gestita direttamente dall'applicazione, introducendo una maggiore complessità. L'uso di pgvector permette invece di affidarsi a un unico database e a un solo `commit`. Inoltre, pgvector è disponibile nei servizi gestiti dei principali fornitori cloud, mantenendo la portabilità della soluzione.

La stessa istanza PostgreSQL gestisce anche la ricerca *full-text* tramite **tsvector**, un tipo che rappresenta il testo in una forma ottimizzata per la ricerca, e gli indici GIN, che rendono più rapide le ricerche sulle parole presenti nel testo. In questo modo, il sistema supporta ricerche relazionali, lessicali e semantiche utilizzando un unico database, senza dover sincronizzare archivi separati.

Garage anziché MinIO. Per l'*object storage*, MinIO è stata inizialmente considerata come soluzione di riferimento. La scelta è ricaduta su Garage anche perché la Community Edition di MinIO non è più mantenuta e il relativo repository è stato archiviato nel 2026. Garage rappresenta quindi un'alternativa open source attivamente sviluppata e, inoltre, è particolarmente adatto al contesto del progetto: è un sistema *masterless* progettato per funzionare su nodi distribuiti e in presenza di connessioni non affidabili. In questo progetto viene utilizzato con un solo nodo, per la ragione infrastrutturale già esposta.

Un ulteriore vantaggio è il supporto all'**API S3**, che mantiene il codice indipendente dall'implementazione utilizzata. Il progetto utilizza infatti **boto3**, lo stesso client impiegato per Amazon S3, e la sostituzione di Garage con un altro servizio compatibile richiederebbe solo la modifica di due variabili d'ambiente.

3.4 Componenti di intelligenza artificiale

Il modello di embedding: Per la ricerca semantica, il sistema utilizza un modello di *embedding* che trasforma il testo di un documento in un vettore numerico. È stato adottato **all-MiniLM-L6-v2**, un modello a 384 dimensioni della famiglia *sentence-transformers*.

La scelta di un modello compatto è motivata dal contesto di esecuzione: deve essere caricato nella memoria del container, condiviso tra tre funzionalità ed eseguito su CPU, senza acceleratori hardware. Modelli da 768 o 1024 dimensioni potrebbero offrire una qualità leggermente superiore, ma con un maggiore consumo di memoria, una latenza più elevata e un indice vettoriale più oneroso. Per un archivio documentale di dimensioni comunali, 384 dimensioni sono quindi sufficienti.

Lo stesso modello è utilizzato per tre funzionalità: generazione degli *embedding*, estrazione delle parole chiave e classificazione della tipologia.

Viene caricato *una sola volta* all'importazione del modulo che lo espone, anziché a ogni richiesta. Non si tratta quindi di una semplice ottimizzazione: il caricamento richiede alcuni secondi e circa 100 MB di memoria, rendendo necessario evitare di ripeterlo per ogni richiesta.

Tesseract anziché un servizio gestito: La traccia propone anche l'utilizzo di servizi cloud per il riconoscimento ottico, come Google Vision e Amazon Textract, generalmente più accurati sulle scansioni di bassa qualità. È stato scelto Tesseract, eseguito localmente nel container, per evitare dipendenze da servizi esterni e dalla connettività, non avere costi per invocazione e, soprattutto, mantenere i documenti all'interno dell'infrastruttura dell'ente. Quest'ultimo aspetto è particolarmente rilevante per atti che possono contenere dati personali.

Il compromesso è una minore accuratezza sulle scansioni degradate, mitigata dalla pre-elaborazione delle immagini e dall'utilizzo del testo riconosciuto come *query* per una ricerca ibrida, anziché come corrispondenza esatta. In questo modo, eventuali errori di riconoscimento possono peggiorare il risultato senza impedirne la ricerca (§4.4.4).

KeyBERT e spaCy: L'estrazione delle parole chiave utilizza KeyBERT, che individua le espressioni più rappresentative confrontando il loro *embedding* con quello dell'intero documento. La scelta è motivata dal fatto che riutilizza il modello già caricato, senza richiedere memoria aggiuntiva significativa. Rispetto ad approcci statistici (come TF-IDF che ignora completamente il contesto in cui una parola viene usata), permette inoltre di individuare espressioni tecniche composte da più parole, come *piano regolatore* e *delibera di giunta*, senza ricorrere a dizionari predefiniti.

Di spaCy viene utilizzato l'elenco delle parole vuote italiane (stop word), necessario perché la libreria sottostante utilizza come impostazione predefinita soltanto l'inglese.

3.5 Frontend

Angular 19 è stato scelto per la struttura che offre a un'applicazione destinata a crescere: tipizzazione statica, iniezione delle dipendenze e componenti autonomi. Il progetto adotta l'approccio *standalone* e il sistema di reattività basato sui *signal*, in cui lo stato del componente è rappresentato da segnali e i valori derivati vengono calcolati a partire da essi. Questo rende esplicite le dipendenze tra i dati e semplifica la gestione dello stato (§6.3).

Nginx serve i file statici prodotti dalla compilazione e funziona anche come *reverse proxy* verso il backend. In questo modo il browser comunica sempre con un'unica origine e non è quindi necessario gestire il problema della condivisione di risorse tra origini diverse (CORS).

4 Scelte Progettuali e Soluzioni

Questo capitolo costituisce il cuore della relazione. Al suo interno sono affrontati i temi centrali dei sistemi distribuiti e viene presentata la forma concreta che hanno assunto in questo progetto: dove risiede lo stato e con quali garanzie di consistenza, come viene governata la concorrenza, come è progettata la ricerca, che cosa accade quando un componente si guasta e come il sistema cresce sotto carico.

4.1 La pipeline di elaborazione documentale

Prima di esaminare i singoli temi, è utile descrivere il percorso che un documento compie nel sistema, perché è su questo flusso che si basano molte delle scelte progettuali successive.

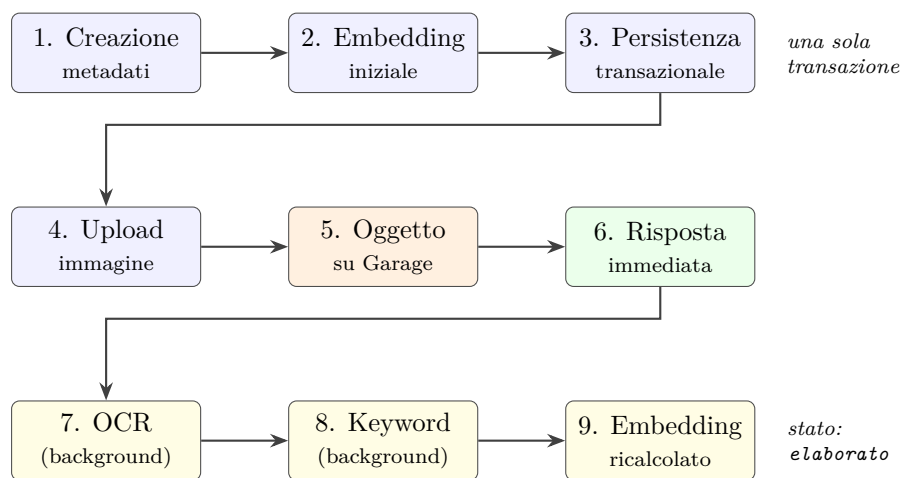


Figura 4.1: Percorso di elaborazione di un documento. I passi 1–3 avvengono nel ciclo richiesta/risposta; i passi 7–9, evidenziati in giallo, sono eseguiti dopo l’invio della risposta al client.

La linea di separazione più importante nella figura è quella tra il passo 6 e il passo 7. In particolare:

- **Sincrono:** tutto ciò che precede il passo 7 deve concludersi in tempi compatibili con l’attesa dell’utente.
- **Asincrono:** tutto ciò che segue il passo 6 viene eseguito in background, perché il riconoscimento ottico di un’immagine ad alta risoluzione può richiedere diversi secondi e non deve occupare il ciclo richiesta/risposta.

Questa scelta riduce la latenza percepita, ma introduce due nuovi problemi: come informare l’utente del completamento e cosa accade se il lavoro in background non termina. Il primo è affrontato in §6.3, il secondo in §4.5.5.

4.2 Gestione dello stato e consistenza

4.2.1 Il teorema CAP e la scelta compiuta

Il teorema CAP (noto anche come teorema di Brewer) stabilisce che un sistema distribuito che conservi dati non può garantire simultaneamente: consistenza, disponibilità e tolleranza

al partizionamento. In presenza di una partizione occorre scegliere fra le prime due.

Per applicare il teorema a questo sistema senza trarre conclusioni errate, è necessaria una precisazione. PostgreSQL è qui un'**istanza singola**: in senso stretto, il teorema non si applica, poiché senza repliche non può verificarsi una partizione tra copie dello stesso dato e il database offre le garanzie ACID di un sistema centralizzato. Il teorema è quindi rilevante non per la configurazione attuale, ma per la sua *direzionale* di evoluzione: introducendo le repliche, la scelta diventerebbe inevitabile ed è opportuno averla considerata in anticipo.

La scelta ricade sulla **consistenza** (CP), per ragioni legate al dominio. In un archivio comunale, un documento che non si aggiorna subito rischia di causare un errore di valutazione: se un ufficio non trova un'ordinanza appena registrata, potrebbe dar per scontato che non esista e comportarsi di conseguenza. Mentre un portale di e-commerce può tollerare un ritardo nel mostrare l'ultimo prodotto aggiunto a catalogo senza conseguenze critiche, un registro pubblico di atti ufficiali esige invece una coerenza immediata e assoluta dei dati.

Per soddisfare questo requisito l'inserimento di un documento e la generazione del suo vettore avvengono nella *medesima transazione*, e non esiste alcun istante in cui un documento sia visibile senza il proprio embedding o in cui il vettore descriva una versione precedente dei metadati.

```

1 def create_documento(db, documento,
2   embedding_precalcolato=_EMBEDDING_NON_FORNITO):
3   db_documento = Documento(**documento.model_dump())
4   if embedding_precalcolato is _EMBEDDING_NON_FORNITO:
5       _aggiorna_embedding(db_documento)      # inferenza del modello
6   else:
7       db_documento.embedding = embedding_precalcolato
8   db.add(db_documento)
9   db.commit()                               # metadati + vettore insieme
10  db.refresh(db_documento)
11  return db_documento

```

Listing 4.1: Creazione di un documento: costruzione dell'entità, generazione del vettore e persistenza avvengono in un'unica transazione (`app/services/documento_service.py`).

Lo stesso principio vale per le modifiche: poiché il vettore rappresenta il significato del documento, una modifica ai campi testuali lo rende obsoleto e richiede un nuovo calcolo.

Il sistema ricalcola però l'*embedding* solo quando necessario: una modifica che riguarda soltanto l'ufficio competente o i firmatari non ne cambia il contenuto e renderebbe inutile eseguire nuovamente il modello. Il codice confronta quindi i campi modificati con quelli utilizzati per generare l'*embedding* e lo ricalcola solo se almeno uno di questi è stato modificato. In questo modo si evita un'elaborazione inutile senza introdurre finestre di inconsistenza.

4.2.2 Consistenza fra due archivi eterogenei

Il vero problema di consistenza distribuita del sistema riguarda il rapporto tra il **database** e l'**object storage**. Associare un'immagine a un documento richiede infatti due scritture (il caricamento dell'oggetto su Garage e l'aggiornamento del riferimento nel database) che avvengono su sistemi separati, privi di un coordinatore transazionale comune. Questo scenario è noto come *dual write*: a prescindere dall'ordine in cui si eseguono le operazioni, un guasto tra la prima e la seconda scrittura lascerebbe il sistema in uno stato parziale.

Una **transazione distribuita a due fasi** risolverebbe formalmente il problema, ma è stata scartata perché sproporzionata rispetto alla natura del dato in gioco. Tale soluzione, infatti, richiederebbe un coordinatore (non supportato da Garage), bloccherebbe temporaneamente entrambi i sistemi e rappresenterebbe essa stessa un nuovo punto di guasto.

La soluzione adottata si basa sull'osservazione che i due possibili stati parziali **non sono equivalenti**:

- un **oggetto orfano** (presente su Garage ma non referenziato da alcuna riga) consuma solo spazio: non compromette alcuna funzionalità, risulta invisibile all'utente e può essere rimosso tramite una semplice scansione periodica;
- un **riferimento pendente** (una riga che punta a un oggetto inesistente) è invece un guasto visibile: l'interfaccia mostrerebbe un'immagine non caricabile accanto a un documento che risulta elaborato correttamente.

L'ordine delle operazioni è stato definito per garantire che, in caso di guasto, il sistema ricada *sempre* nel primo stato (quello dell'**oggetto orfano**).

Durante il **caricamento**, l'oggetto viene salvato su Garage *prima* dell'aggiornamento nel database: se il caricamento fallisce, l'operazione si interrompe senza conseguenze; se fallisce l'aggiornamento successivo, l'oggetto diventa orfano.

Durante l'**eliminazione**, l'ordine è invertito e l'errore non viene deliberatamente propagato:

```

1 if db_documento.immagine_url:
2     try:
3         garage_service.delete_file(db_documento.immagine_url)
4     except Exception as e:
5         # Un oggetto orfano su Garage e' preferibile a un'eliminazione
6         # negata all'utente. Il log e' pero' indispensabile: senza,
7         # un guasto sistematico dello storage resterebbe invisibile.
8         logger.warning(
9             "Impossibile cancellare l'immagine '%s' per il documento %d: %s",
10            db_documento.immagine_url, documento_id, e)
11
12 documento_service.delete_documento(db, documento_id)

```

Listing 4.2: Eliminazione di un documento: il fallimento della cancellazione dell'oggetto non impedisce quella della scheda (`app/routers/documento_controller.py`).

Tracciare queste anomalie nei log è però essenziale. Assorbire l'errore in silenzio renderebbe il sistema solo apparentemente più robusto, ma molto meno monitorabile (eventuali cancellazioni rifiutate da Garage farebbero accumulare oggetti orfani di nascosto). La strategia adottata garantisce continuità e trasparenza verso l'utente, evitando al contempo che qualsiasi anomalia venga nascosta all'operatore.

La stessa logica viene applicata anche alla sostituzione di un'immagine esistente.

4.2.3 Stato derivato materializzato nello schema

Durante lo sviluppo è emersa un'ambiguità sullo stato di elaborazione di un documento (*in_attesa*, *elaborato*, *errore*). Il valore iniziale imposto dal database (*in_attesa*) descrive infatti due situazioni distinte: un'elaborazione realmente in corso oppure una scheda inserita con i soli metadati (priva di immagine e per cui nessuna elaborazione è mai partita).

Per distinguere i due casi occorre combinare due colonne. Inizialmente questa logica era replicata in due punti (in SQL per le statistiche e in TypeScript per l'interfaccia), con il rischio concreto di creare divergenze se modificata solo da una parte.

La soluzione adottata materializza questa regola *nello schema*, come colonna generata e memorizzata direttamente in PostgreSQL:

```

1 stato_effettivo = Column(
2     String(50),
3     Computed(
4         "CASE WHEN immagine_url IS NULL "
5         "THEN 'senza_scansione' ELSE stato_elaborazione END",
6         persisted=True,
7     ),
8     nullable=False,
9 )

```

Listing 4.3: Colonna generata che risolve l'ambiguità alla fonte (`app/models/documento.py`).

Il valore calcolato diventa così parte del dato stesso anziché della sua interpretazione. Le statistiche si semplificano e il frontend riceve un'informazione già pronta. Il vantaggio principale non è prestazionale, ma risiede nell'eliminazione di possibili incoerenze: quando più componenti scritti in linguaggi diversi devono interpretare la stessa informazione, spostare la logica nel database è il modo più sicuro per garantire coerenza.

4.2.4 Idempotenza e riconciliazione

Due meccanismi consentono al sistema di tornare a uno stato coerente dopo un'anomalia.

Il primo è l'**idempotenza dell'inizializzazione**: la procedura che crea schema, indici e *trigger* viene eseguita a ogni avvio e deve poter essere ripetuta senza conseguenze (le tabelle sono create solo se assenti, la funzione dell'indice full-text è ridefinita con `CREATE OR REPLACE`, il *trigger* è rimosso e ricreato). Un test specifico verifica che invocazioni consecutive non producano effetti distruttivi.

Il secondo meccanismo è la **riconciliazione dei vettori mancanti**. Un documento può risultare privo di embedding per ragioni legittime (ad esempio a causa di un'elaborazione fallita); in questo stato, partecipa solo alla ricerca full-text, restando parzialmente invisibile. Un endpoint di manutenzione individua le righe interessate e ne ricalcola il vettore. L'operazione è idempotente (agisce solo dove l'embedding è assente) e produce un vantaggio collaterale: la scrittura attiva il *trigger* che aggiorna anche l'indice full-text, riallineando in un solo passaggio entrambe le modalità di ricerca.

4.3 Concorrenza

4.3.1 Il modello di esecuzione

FastAPI offre due modalità di esecuzione per le richieste, una distinzione che rappresenta una delle fonti di errore più insidiose del framework. Una funzione dichiarata `async def` viene eseguita *nel ciclo di eventi* condiviso, mentre una funzione dichiarata `def` gira in un *pool di thread* separato. La regola fondamentale è semplice: nel ciclo di eventi non deve esserci alcuna chiamata bloccante, altrimenti nessun'altra richiesta potrà progredire.

Nel progetto, le funzioni che eseguono I/O tramite librerie sincrone (come boto3 per il client S3 verso Garage e SQLAlchemy) o calcoli intensivi su CPU sono dichiarate esclusivamente

come `def`. In questo modo, l'esecuzione viene demandata al pool di thread di FastAPI. Ad esempio, per l'endpoint di caricamento, ciò garantisce che il trasferimento dei dati non congeli le altre richieste, permettendo alla risposta di tornare subito e al compito in background di avviarsi correttamente. L'uso di `async def` sarebbe appropriato solo per librerie di I/O asincrone, che l'applicazione attualmente non impiega.

4.3.2 Conflitti di inizializzazione dello schema del DB

Il difetto di concorrenza più significativo del progetto si manifesta soltanto quando il backend viene replicato (invisibile su un'istanza singola).

All'avvio, ciascuna replica esegue la procedura di inizializzazione dello schema del database. Poiché la creazione delle tabelle non è atomica (ispeziona il catalogo per verificare le tabelle esistenti e solo dopo crea quelle mancanti), si crea una finestra temporale critica. Con l'avvio simultaneo di più repliche, tutte osservano un catalogo vuoto e tentano la creazione, generando errori per violazione di unicità sulla sequenza degli identificatori.

La soluzione impiega un **lock consultivo** di PostgreSQL, un meccanismo di mutua esclusione valido fra connessioni distinte (e quindi fra processi e container).

```

1 with engine.connect() as conn:
2     conn.execute(text("SELECT pg_advisory_lock(:key)"),
3                     {"key": _SCHEMA_INIT_LOCK_KEY})
4     try:
5         Base.metadata.create_all(bind=conn)    # sezione critica
6         crea_trigger_search_vector(conn)
7         conn.commit()
8     finally:
9         # Il rollback precede il rilascio: dopo un errore PostgreSQL
10        # rifiuta ogni altro comando finché la transazione non è
11        # annullata, e il lock resterebbe acquisito.
12        conn.rollback()
13        conn.execute(text("SELECT pg_advisory_unlock(:key)"),
14                      {"key": _SCHEMA_INIT_LOCK_KEY})
15        conn.commit()

```

Listing 4.4: Serializzazione dell'inizializzazione dello schema fra repliche concorrenti (app/database.py, estratto).

Due dettagli sono cruciali per il corretto funzionamento. Il primo è il rilascio del lock in un blocco `finally`: se un errore interrompesse la sezione critica senza rilasciarlo, le altre repliche resterebbero bloccate all'infinito. Il secondo è l'annullamento della transazione *prima* del rilascio: dopo un errore, PostgreSQL rifiuta ogni comando finché la transazione non viene annullata (senza questo passaggio anche il comando di rilascio fallirebbe).

La procedura è inoltre racchiusa in un ciclo di tentativi per assorbire le indisponibilità transitorie del database all'avvio; esauriti i tentativi, l'errore viene propagato e l'applicazione non si avvia.

4.3.3 Contesa di risorse fra elaborazioni simultanee

Un problema di concorrenza legato al **motore OCR** è stato anticipato analizzandone il funzionamento. Poiché la sua libreria di parallelizzazione apre un thread per core per ogni immagine elaborata in background, il rischio di sovraccarico era evidente: poche operazioni simultanee avrebbero generato un numero di thread nettamente superiore ai core effettivi. Il

sistema sarebbe così entrato in forte *contesa* (con tempi di commutazione tra thread superiori a quelli di reale elaborazione).

Questa intuizione teorica è stata successivamente messa alla prova e confermata tramite un test di carico mirato, che ha puntualmente registrato il previsto crollo delle prestazioni.

Scenario	Senza limite	Con limite a 1 thread
Un solo OCR isolato	circa 0,70 s	circa 1,20 s
Otto OCR simultanei (totale)	oltre 70 s	circa 1,90 s

Tabella 4.1: Effetto della limitazione dei thread del motore OCR sui tempi di elaborazione.

Il compromesso è favorevole: limitare i thread rallenta la singola elaborazione di circa mezzo secondo, ma migliora il comportamento sotto carico concorrente di oltre un ordine di grandezza (è preferibile un modesto rallentamento a sistema scarico piuttosto che il collasso con più operatori simultanei).

La limitazione viene applicata a **tempo di esecuzione** e non come variabile d'ambiente, perché la stessa variabile è letta anche dalla libreria di calcolo numerico dei modelli al momento della propria inizializzazione, e in quella configurazione la prima elaborazione OCR resta bloccata a tempo indefinito. Impostandola a tempo di esecuzione, quando l'ambiente di calcolo è già inizializzato, il limite viene ereditato soltanto dai sottoprocessi OCR successivi, senza alterare le prestazioni di embedding ed estrazione delle parole chiave.

4.3.4 Gestione thread-safe della cache dei prototipi

Un ultimo caso riguarda il classificatore di tipologia, che confronta il testo di un documento con i vettori di otto prototipi testuali. Tali vettori sono costanti e vengono calcolati alla prima richiesta anziché all'avvio, per non gravare su un tempo di partenza già occupato dal caricamento dei modelli. Poiché l'endpoint è eseguito nel pool di thread, più richieste possono raggiungere il calcolo con la cache ancora vuota: la protezione adottata è il *double-checked locking*, con la verifica effettuata una prima volta senza lock e ripetuta al suo interno.

È importante però precisare che l'assegnazione finale del dizionario è atomica e nessun thread può osservarne una versione parziale. Questo è quindi un lock a tutela dell'**efficienza** e non della correttezza: evita semplicemente che otto operazioni vengano eseguite in parallelo per produrre lo stesso identico risultato.

4.4 Progettazione della ricerca

La ricerca è la funzionalità che maggiormente caratterizza il sistema, ed è realizzata in quattro modalità complementari.

4.4.1 Ricerca full-text

La ricerca lessicale impiega il tipo `tsvector` di PostgreSQL, che rappresenta un documento come insieme di parole normalizzate. La colonna non è popolata dall'applicazione ma da un *trigger* del database, invocato a ogni inserimento e aggiornamento: collocare la logica nel database garantisce che l'indice resti allineato *qualunque* sia il percorso della scrittura, inclusi gli aggiornamenti massivi eseguiti direttamente in SQL, che un aggiornamento affidato all'applicazione non intercetterebbe. Il *trigger* attribuisce inoltre pesi differenziati ai campi

(nome, descrizione, tipologia e infine testo riconosciuto) secondo una graduatoria che riflette l'affidabilità della fonte: il nome è inserito da un operatore ed è deliberatamente descrittivo, mentre il testo OCR può contenere errori di riconoscimento e pesa quindi meno di ogni altro.

```

1 NEW.search_vector :=
2   setweight(to_tsvector('italian', coalesce(NEW.nome, '')), 'A') ||
3   setweight(to_tsvector('italian', coalesce(NEW.descrizione, '')), 'B') ||
4   setweight(to_tsvector('italian', coalesce(NEW.tipologia, '')), 'C') ||
5   setweight(to_tsvector('italian', coalesce(NEW.testo_ocr, '')), 'D');
```

Listing 4.5: Trigger di aggiornamento dell'indice full-text con pesi differenziati (app/database.py).

Sulla combinazione delle parole, invece, la scelta più immediata sarebbe stata quella di combinarli in *congiunzione*, restituendo i soli documenti che le contengono tutte; si è preferita la *disgiunzione*, costruendo un'interrogazione per ciascuna parola e unendole con l'operatore di alternativa, così che una ricerca per *traffico commercio* restituisca i documenti contenenti almeno una delle due parole. Senza questa soluzione, la congiunzione avrebbe prodotto con frequenza risultati vuoti; inoltre con la soluzione adottata l'ordinamento per rilevanza colloca comunque in testa i documenti che soddisfano entrambi i termini.

L'interrogazione è sostenuta da un indice GIN, in assenza del quale il database dovrebbe scandire sequenzialmente l'intera tabella.

4.4.2 Ricerca semantica

La ricerca semantica genera l'embedding della query e individua i documenti più prossimi tramite distanza coseno (ridotta al prodotto scalare per i vettori normalizzati). Il modello mappa testi di significato affine in punti vicini di uno spazio a 384 dimensioni, permettendo di trovare corrispondenze anche senza parole in comune (ad esempio, la query *regolamento per costruire una casa* individua il *Piano Regolatore Edilizio*).

L'interrogazione sfrutta un indice HNSW, una struttura a grafo per la ricerca *approssimata* in tempo sub-lineare. L'indice non garantisce il recupero esatto dei risultati migliori, ma rappresenta il compromesso ideale in un contesto dove i risultati fungono da suggerimenti per la valutazione di un operatore umano.

Il punteggio finale viene trasformato in una similarità compresa tra 0 e 1, strettamente vincolata a non assumere valori negativi. Questa normalizzazione è essenziale: poiché la distanza coseno appartiene all'intervallo $[0, 2]$, una similarità negativa corromperebbe le classifiche finali qualora venisse sommata ai punteggi della ricerca lessicale.

4.4.3 Ricerca ibrida

Le due modalità precedenti hanno punti di forza complementari: quella lessicale è accurata sulle corrispondenze letterali forti (un numero di protocollo, una data, un codice) mentre quella semantica coglie relazioni che nessuna corrispondenza di stringhe rivelerebbe. La modalità ibrida le combina in un unico punteggio pesato:

$$score = (1 - w) \cdot \frac{score_{ft}}{\max(score_{ft})} + w \cdot score_{sem}$$

dove $w \in [0, 1]$ è il peso attribuito alla componente semantica, configurabile dal client e predefinito a 0,5.

La normalizzazione del primo addendo è necessaria: la funzione di rilevanza di PostgreSQL non restituisce un valore in un intervallo prefissato, e sommarla direttamente a una similarità coseno significherebbe sommare quantità disomogenee.

Questa normalizzazione viene calcolata rispetto al punteggio massimo della *singola ricerca*. I punteggi risultano così confrontabili solo all'interno degli stessi risultati (una limitazione accettabile, dato che servono solo per l'ordinamento relativo). Infine, ciascuna modalità estrae un numero di risultati doppio rispetto a quello richiesto: un documento potrebbe infatti eccellere in una sola delle due ricerche, ma meritare comunque di comparire nella classifica finale combinata.

4.4.4 Ricerca per immagine

La quarta modalità permette di cercare la scheda di un atto cartaceo senza digitare nulla. L'operatore fotografa il documento e il sistema ne estrae il testo tramite OCR, utilizzandolo come query per una ricerca ibrida.

L'impiego della modalità ibrida è la scelta chiave e discende dalla natura del testo riconosciuto, che conterrà quasi certamente errori: la componente lessicale intercetta le corrispondenze esatte sopravvissute al riconoscimento (come numeri di protocollo, date e nomi di uffici), mentre la componente semantica tollera le imprecisioni (un testo con qualche carattere errato mantiene un vettore molto simile a quello corretto). Le due si compensano proprio nel punto in cui ciascuna è debole.

L'immagine caricata funge solo da query temporanea e non viene salvata. Tuttavia, il sistema restituisce il testo effettivamente riconosciuto per mostrarlo all'operatore: in caso di esiti insoddisfacenti, leggere che cosa ha interpretato l'OCR è l'unico modo per distinguere un documento mancante da una cattiva scansione. Infine, se non viene estratto alcun testo utile, il sistema restituisce semplicemente un elenco vuoto anziché un errore.

4.4.5 Documenti simili e suggerimento della tipologia

Dallo stesso apparato vettoriale derivano due funzionalità aggiuntive.

La prima propone, nella scheda di dettaglio, i documenti semanticamente più prossimi a quello consultato, confrontando il vettore già memorizzato con quelli degli altri documenti ed escludendo le righe prive di embedding.

La seconda suggerisce la tipologia amministrativa in fase di inserimento, confrontando il testo della scheda con otto prototipi testuali che descrivono le classi tipiche e restituendo le più probabili ordinate per confidenza.

L'approccio a prototipi è stato preferito a un classificatore per la mancanza di un dataset di documenti comunali già etichettato (crearne uno esulerebbe dal progetto). Questo metodo non richiede addestramento, riutilizza il modello esistente ed è facilmente estendibile tramite l'aggiornamento di un dizionario. Il suggerimento fornito, inoltre, è deliberatamente *non vincolante*: poiché la classificazione di un atto ha rilevanza giuridica, una decisione autonoma del sistema risulterebbe del tutto inappropriata.

4.5 Tolleranza ai guasti

In un sistema distribuito, il guasto parziale non è un'eccezione ma la norma. La questione progettuale non è *se* un componente fallirà, ma come reagirà il sistema. Questa sezione illustra

proprio le contromisure previste per ciascuna classe di guasto.

4.5.1 Tassonomia dei guasti e risposte adottate

Guasto	Rilevazione	Risposta del sistema
Database non pronto all'avvio	Sonda di prontezza con verifica di connessione effettiva	Il backend non viene avviato finché la sonda non ha esito positivo
Database irraggiungibile in esercizio	Verifica preventiva delle connessioni del pool; query minima nella sonda del backend	Riapertura trasparente della connessione; se persiste, il backend si dichiara non sano
Storage non pronto all'avvio	Sonda semantica sulla disponibilità del bucket	Avvio del backend posticipato
Storage irraggiungibile in eliminazione	Eccezione del client S3	Errore registrato, operazione completata: si accetta un oggetto orfano (§4.2.2)
Storage lento o non responsivo	Timeout espliciti di connessione e lettura	La chiamata fallisce rapidamente anziché occupare a lungo un thread
OCR fallito su un'immagine	Testo estratto vuoto	Stato portato a errore ; la scheda resta valida e ricercabile per metadati
Motore OCR assente	Eccezione specifica del binario mancante	Registrata con severità critica e distinta da un'immagine illeggibile: guasto di configurazione, non esito ordinario
Riga non valida in importazione	Errore di validazione o del database	Annullamento della sola riga, prosecuzione del lotto, rapporto analitico
Corsa critica sullo schema db	—	Prevenuta tramite lock consultivo (§4.3.2)
Arresto anomalo di un container	Politica di riavvio dell'orchestratore	Riavvio automatico; lo stato è preservato dai volumi

Tabella 4.2: Guasti previsti, modalità di rilevazione e risposta del sistema.

4.5.2 Disponibilità effettiva del servizio e ordine di avvio

L'avvio di uno stack a più servizi richiede una precisa sincronizzazione.

La soluzione più banale non funziona: far dipendere il backend dal database garantisce solo che il container sia stato *avviato*, ma non che il servizio accetti connessioni (tra questi due eventi c'è un intervallo in cui ogni tentativo di accesso fallisce). Per questo, l'avvio del sistema è condizionato a una sonda di prontezza che verifica la reale disponibilità *semantica* del servizio, anziché la semplice presenza del processo.

Per garantire una corretta sincronizzazione all'avvio, ogni componente dello stack implementa una specifica sonda. La Tabella 4.3 illustra i controlli adottati per assicurarsi che i servizi siano realmente operativi, andando oltre la semplice verifica del processo.

Servizio	Sonda e ragione della sua formulazione
db	Verifica di accettazione delle connessioni sull'interfaccia di rete (e non sul socket locale). Senza tale precisazione, il database si dichiarerebbe pronto durante l'esecuzione dello script di inizializzazione, avviando il backend troppo presto.
garage	Interrogazione delle informazioni del bucket (e non semplice verifica di risposta del processo). L'esito è positivo soltanto quando la struttura interna e le credenziali sono state effettivamente predisposte.
backend	Interrogazione dell'endpoint dedicato, che a sua volta esegue una query minima sul database. Un processo attivo ma isolato dal database risulterebbe altrimenti sano, mentre ogni funzionalità reale fallirebbe.

Tabella 4.3: Sonde di prontezza e motivazione della loro formulazione.

La sonda del backend ne è un ottimo esempio: inizialmente rispondeva sempre in modo positivo (confermando solo che il processo fosse in esecuzione, un'informazione già nota all'orchestratore). Ora, eseguendo una query minima, verifica che il servizio sia effettivamente in grado di *fare* il proprio lavoro.

Due parametri di questa configurazione meritano attenzione. Il periodo di grazia iniziale è impostato a 90 secondi per consentire il caricamento del modello vettoriale e delle risorse linguistiche (una tolleranza inferiore scambierebbe un'inizializzazione legittima per un guasto, innescando un ciclo infinito di riavvii). Durante questo lasso di tempo, però, la frequenza di interrogazione viene aumentata: in questo modo l'avvio si conclude non appena il servizio è pronto, senza dover attendere il normale intervallo di controllo.

4.5.3 Strategie di retry e controllo dei timeout

Il sistema adotta la ripetizione dei tentativi in tre scenari specifici, ma mai in modo indefinito:

- l'**inizializzazione dello schema** (ritentata fino a tre volte per assorbire le indisponibilità transitorie all'avvio);
- le **chiamate allo storage** (affidate alla politica standard del client S3, con un massimo di tre tentativi);
- le **connessioni al database** (verificate e riaperte trasparentemente se chiuse dalla controparte, evento frequente se il database viene riavviato).

Altrettanto cruciali sono i timeout espliciti imposti al client S3 (5 secondi per la connessione e 20 per la lettura, anziché i 60 predefiniti). Poiché queste chiamate sono sincrone all'interno delle richieste HTTP e Garage si trova sulla stessa rete interna (con latenza attesa sotto il millisecondo), un timeout permissivo non renderebbe il sistema più tollerante, ma solo più lento. Uno storage non responsivo finirebbe per bloccare i thread per un intero minuto, esaurendo rapidamente il pool a disposizione. Riconoscere subito un guasto, infatti, è anche importante.

4.5.4 Continuità operativa in presenza di errori parziali

Il principio fondamentale è che il guasto di un singolo componente non deve compromettere l'intero sistema. Ecco alcune applicazioni concrete:

- Nell'**importazione massiva**, il fallimento di una singola riga non interrompe l'intero lotto. La transazione viene annullata solo per il record interessato e l'errore viene registrato con il numero di riga (permettendone la facile individuazione). Il rapporto finale riepiloga successi e fallimenti. È preferibile importare 298 righe su 300 e segnalare le due errate, anziché rifiutare l'intero file.
- Nella **dashboard**, un errore nel calcolo delle statistiche non impedisce di visualizzare gli ultimi documenti inseriti (i contatori restano a zero, ma il resto della pagina funziona).
- Nella **scheda di dettaglio**, l'indisponibilità dei suggerimenti semantici non blocca la consultazione del documento. Se l'immagine non si carica, viene mostrato un segnaposto (che distingue un'effettiva assenza di scansione dalla sua indisponibilità temporanea).
- Nell'**elaborazione in background**, un'eccezione non lascia il documento in uno stato indeterminato. La transazione viene prima annullata (passaggio necessario per non bloccare per sempre le query successive) e lo stato passa a **errore**, rendendo l'anomalia visibile all'operatore anziché assorbirla in silenzio.

4.5.5 Limiti noti della tolleranza ai guasti

La resilienza del sistema ha confini precisi, che è essenziale dichiarare:

I compiti in background non sono durabili: I task differiti di FastAPI vivono solo nel processo che li crea. Se il backend si arresta durante l'elaborazione OCR, il lavoro va perso (lasciando il documento bloccato indefinitamente in stato `in_attesa`). Questa è la contropartita diretta della scelta di non usare un broker di messaggi (che invece garantirebbe la riconsegna al riavvio). L'endpoint di riconciliazione rimedia solo ai vettori mancanti, ma non riesegue l'OCR.

Assenza di interruttore di circuito: Non essendo implementato un *circuit breaker*, se Garage diventa persistentemente non responsivo le richieste continuano a tentare la chiamata e a fallire dopo il timeout (consumando inutilmente thread). I timeout brevi limitano il danno, ma per questa scala architetturale il costo di un componente aggiuntivo non sarebbe ripagato.

Punti singoli di guasto: Database e storage sono istanze uniche e la loro perdita rende il sistema inutilizzabile, per la ragione infrastrutturale esposta in §1.5. La direzione di superamento è indicata in §4.6.2.

4.6 Scalabilità

4.6.1 Scalabilità orizzontale del livello applicativo

L'elasticità richiede che le unità di elaborazione siano prive di stato locale (una condizione che il backend soddisfa, come esposto in §2.4). Per sfruttare realmente questa proprietà è stato però necessario rimuovere un vincolo di configurazione.

Inizialmente, il file di orchestrazione assegnava a ogni servizio un nome esplicito. Essendo questi nomi globali nel *daemon* Docker, l'applicazione era istanziabile una sola volta per host (causando conflitti all'avvio di più repliche o di un secondo ambiente parallelo).

Rimuovendo l'attributo, l'orchestratore genera ora nomi dinamici (secondo lo schema `<progetto>-<servizio>-<indice>`), pur mantenendo l'accesso operativo tramite il nome stabile del *servizio*. Questo caso dimostra chiaramente come una banale scelta di configurazione possa vanificare un'intera proprietà architettuale.

Rimosso il vincolo, la scalatura è stata testata con questo risultato:

```

1 $ docker compose up -d --scale backend=3
2
3 NAME                SERVICE    STATUS
4 sdcc-backend-1      backend    Up 5 seconds (healthy)
5 sdcc-backend-2      backend    Up 5 seconds (healthy)
6 sdcc-backend-3      backend    Up 5 seconds (healthy)
7 sdcc-db-1           db         Up About a minute (healthy)
8 sdcc-garage-1        garage     Up About a minute (healthy)
9 sdcc-frontend-1     frontend   Up 28 seconds

```

Listing 4.6: Replicazione del livello applicativo e stato risultante.

Le tre repliche superano in modo indipendente la propria sonda di prontezza, condividendo i medesimi servizi di persistenza. Il risolutore DNS interno associa all'unico nome logico i loro indirizzi in ordine variabile, permettendo al *reverse proxy* di distribuire le richieste in logica *round-robin*. Questo test ha peraltro reso osservabile la corsa critica descritta in §4.3.2.

Il bilanciamento tramite DNS è rudimentale: i client ne memorizzano l'indirizzo, il sistema ignora il carico reale delle singole repliche e, soprattutto, continua a inviare traffico anche alle istanze guaste. In produzione serve un vero bilanciatore applicativo che escluda attivamente i servizi bloccati (funzionalità offerta nativamente dagli orchestratori avanzati in §??).

4.6.2 Colli di bottiglia

Replicare solo il livello applicativo sposta il collo di bottiglia sui servizi di persistenza, che partono da presupposti diversi.

Garage è nativamente un sistema distribuito *masterless* (qui limitato a un singolo nodo). La sua estensione a cluster non richiederebbe modifiche applicative: il backend continuerebbe a interrogare lo stesso endpoint S3, delegando allo storage la distribuzione dei dati.

PostgreSQL è invece il vero componente critico. Per potenziarlo si userebbe la **replica in lettura** (ideale in un sistema dove le ricerche superano di gran lunga le scritture). Questo approccio introduce però un leggero ritardo nell'aggiornamento dei dati (la cosiddetta **consistenza finale**, coerentemente con il quadro CAP discusso in §4.2.1): un documento appena salvato potrebbe non apparire subito se l'utente interroga una replica non ancora allineata. Poiché i requisiti di progetto vietano questo comportamento, bisognerebbe forzare l'applicazione a leggere dal database principale (sempre aggiornato) subito dopo una scrittura. È un compromesso tecnico molto valido, ma che richiede di essere progettato su misura.

4.6.3 Ottimizzazioni adottate

Calcolo degli embedding a lotti: L'approccio iniziale (più semplice ma inefficiente) prevedeva di invocare il modello vettoriale una riga alla volta durante l'importazione massiva. Ogni singola chiamata, tuttavia, comportava un costo fisso ineliminabile (la preparazione dei tensori e il loro trasferimento al motore di calcolo).

Raggruppando l'operazione in un unico lotto prima del ciclo di inserimento, questo costo viene ammortizzato sull'intera lista, accelerando l'elaborazione. Questa ottimizzazione è del

tutto trasparente: i vettori ottenuti sono identici a quelli dell’elaborazione singola. Inoltre, i testi vuoti (che non necessitano di calcolo) vengono temporaneamente esclusi dal lotto e poi reinseriti nella loro posizione originale (preservando così il corretto allineamento delle righe senza causare errori).

Statistiche calcolate dal database: All’inizio la dashboard ricavava i contatori dai dati paginati dell’API (ottenendo totali errati e bloccati a 100 elementi). Creando un endpoint dedicato per l’aggregazione diretta in SQL, il problema è stato risolto. Delegare il calcolo al database non ha solo ottimizzato le prestazioni (trasferendo unicamente numeri anziché intere righe), ma ha soprattutto garantito la *correttezza* del risultato.

Indici. Le due modalità di ricerca sono sostenute da indici dedicati dichiarati nella definizione del modello: GIN sulla colonna full-text, HNSW sulla colonna vettoriale. In assenza del secondo, ogni ricerca semantica richiederebbe il calcolo della distanza fra la query e ogni vettore dell’archivio.

4.7 Sicurezza

Diverse misure di sicurezza sono state adottate a livello infrastrutturale, secondo il principio del minimo privilegio. La Tabella 4.4 riassume le principali contromisure adottate.

Misura di sicurezza	Descrizione e motivazione
Segmentazione della rete	Database e storage risiedono solo sulla rete interna (senza porte sull’host). Il backend fa da unico ponte: un attacco al frontend non offre alcun accesso diretto ai dati.
Esecuzione senza privilegi	I container girano con un utente non amministrativo e i compilatori sono rimossi dall’immagine finale, riducendo i rischi in caso di esecuzione di codice arbitrario.
Superficie esposta minima	Di Garage è pubblica solo la porta S3 per il browser; la console di amministrazione e il server web integrato sono isolati nella rete privata.
URL firmati per gli oggetti	Le immagini non sono pubbliche: il browser accede tramite link a scadenza (1 ora) generati dal backend [1]. Per gestire il cambio di indirizzo tra rete interna e client, il sistema usa due client S3 distinti.
Segreti esterni	Nessuna credenziale è hardcodata. Variabili d’ambiente obbligatorie bloccano l’avvio se mancanti, evitando valori predefiniti insicuri.
Validazione degli ingressi	Ogni input è validato in modo dichiarativo al confine dell’API (es. limiti di 10 MB sulle immagini e 5 MB sull’importazione), impedendo dati anomali a valle.

Tabella 4.4: Misure di sicurezza infrastrutturale e motivazioni.

5 Deployment e Infrastruttura Cloud

5.1 Virtualizzazione a livello di sistema operativo

La containerizzazione è una forma di virtualizzazione a livello di sistema operativo: i container condividono il kernel dell'host e sono isolati fra loro tramite i meccanismi che il kernel stesso fornisce (spazi dei nomi per l'isolamento della visibilità e gruppi di controllo per la ripartizione delle risorse). È una differenza sostanziale rispetto alla virtualizzazione tradizionale, in cui ciascuna macchina virtuale esegue un proprio kernel al di sopra di un *hypervisor*.

	Macchina virtuale	Container
Isolamento	Completo: kernel dedicato	Di processo: kernel condiviso
Avvio	Decine di secondi	Sotto il secondo
Dimensione	Ordine dei gigabyte	Ordine dei megabyte (esclusi i dati)
Densità per host	Unità o decine	Centinaia
Portabilità	Legata al formato dell'immagine	Immagine standard OCI

Tabella 5.1: Confronto fra virtualizzazione completa e containerizzazione.

Sebbene in questo progetto la scelta della containerizzazione fosse obbligata dalla traccia, si sarebbe rivelata comunque la più appropriata.

5.2 Costruzione dell'immagine del backend

Il file di costruzione dell'immagine del backend concentra diverse scelte progettuali, ciascuna con una motivazione precisa.

Immagine di base ancorata al digest: L'immagine di partenza è indicata tramite il suo *digest* crittografico e non tramite un'etichetta mobile. Un'etichetta come `python:3.11-slim` designa contenuti diversi in momenti diversi, venendo riassegnata a ogni aggiornamento: due costruzioni a distanza di settimane produrrebbero ambienti differenti a parità di sorgenti. Il digest identifica un contenuto immutabile: gli aggiornamenti della base diventano così scelte esplicite e tracciabili.

Ordinamento dei livelli per la cache: Il file delle dipendenze è copiato e installato *prima* del codice sorgente. Poiché ogni istruzione produce un livello memorizzato nella cache e invalidato solo quando i file da cui dipende cambiano, una modifica al codice non invalida l'installazione delle dipendenze (operazione più costosa). Invertire le due istruzioni non cambierebbe il risultato ma renderebbe ogni costruzione lunga quanto la prima.

Ambiente di calcolo senza supporto per acceleratori: La libreria utilizzata per il calcolo tensoriale, cioè l'elaborazione di dati multidimensionali, è installata nella variante dedicata alle sole CPU. La versione predefinita richiede infatti 29 pacchetti per 2,82 GiB, di cui 2,32 GiB destinati al supporto delle schede grafiche, inutilizzabile in assenza di acceleratori. Si riducono così spazio, dipendenze e superficie di attacco.

Modelli scaricati in fase di costruzione: I pesi dei modelli sono scaricati durante la costruzione, evitando download al primo avvio. Il container può quindi partire senza connettività, senza dipendere dalla banda e senza problemi di permesso sulla cache. La contropartita è una maggiore dimensione dell'immagine.

Rimozione degli strumenti di compilazione: Il compilatore C, necessario a costruire il driver del database, è rimosso nella stessa istruzione che lo installa. Rimuoverlo successivamente lascerebbe infatti il pacchetto in un livello sottostante dell'immagine. Viene infine creato un utente dedicato, utilizzato per l'avvio del processo secondo il principio discusso in §4.7.

Il processo applicativo come processo iniziale. Il comando di avvio *sostituisce* la shell con il server:

```
1  
2 CMD ["sh", "-c", "exec uvicorn app.main:app --host 0.0.0.0 --port 8000  
    ${UVICORN_EXTRA_ARGS:-}"]
```

Listing 5.1: Comando di avvio del container: la sostituzione del processo consente una terminazione pulita.

Il segnale di terminazione raggiunge infatti il processo con identificatore 1. Senza la sostituzione, tale processo sarebbe la shell e il container richiederebbe la terminazione forzata allo scadere del timeout. Questa scelta fa la differenza nei riavvii frequenti e consente inoltre di chiudere ordinatamente le connessioni.

5.3 Costruzione del frontend

L'immagine del frontend impiega una costruzione a due stadi:

- il primo contiene l'ambiente JavaScript necessario a compilare l'applicazione,
- mentre il secondo include soltanto il server web e i file prodotti dalla compilazione.

L'immagine distribuita non contiene quindi ambiente di sviluppo, dipendenze di compilazione o codice sorgente, ma soltanto file statici e il server che li serve. Ne derivano una dimensione notevolmente inferiore e una superficie di attacco ridotta, poiché l'assenza di interprete e gestore di pacchetti limita le risorse disponibili a chi ottenesse accesso al container. L'installazione rispetta inoltre rigorosamente il file di blocco delle versioni, garantendo che costruzioni successive installino gli stessi pacchetti.

5.4 Orchestrazione con Docker Compose

Il file di orchestrazione descrive in forma dichiarativa lo stato desiderato del sistema. Oltre alle sonde di prontezza già discusse in §4.5.2, incorpora alcune scelte rilevanti.

Variabili obbligatorie. Le variabili prive di un valore predefinito rendono l'avvio impossibile in loro assenza, mostrando un messaggio esplicativo. È preferibile un errore immediato a un valore predefinito per una credenziale, che farebbe fallire il sistema solo successivamente e lontano dalla causa.

Rotazione dei registri. La configurazione dei registri è definita una sola volta tramite un'ancora YAML e applicata a tutti i servizi, limitando la conservazione a tre file da 10 MB. Senza tale limite i registri crescerebbero indefinitamente fino a saturare il disco dell'host.

Politica di riavvio. Tutti i servizi sono configurati per il riavvio automatico salvo arresto esplicito: un container terminato per errore viene riavviato, mentre uno arrestato deliberatamente non viene riavviato dal *daemon*.

Assenza di nomi di container espliciti. Come discusso in §4.6.1, i nomi sono lasciati derivare all'orchestratore per non precludere la replicazione.

5.5 Segmentazione della rete

Le due reti virtuali definite realizzano una separazione a tre livelli. La Tabella 5.2 riporta l'appartenenza di ciascun servizio e le porte pubblicate verso l'host.

Servizio	frontend-net	backend-net	Porte pubblicate
frontend	✓	—	80 (HTTP)
backend	✓	✓	8000 (API)
db	—	✓	nessuna
garage	—	✓	3900 (sola API S3)

Tabella 5.2: Appartenenza dei servizi alle reti e porte esposte verso l'host.

L'assenza di porte pubblicate per il database è voluto: il servizio è raggiungibile solo dall'interno della rete privata, e nessuna configurazione errata del firewall dell'host può esporlo. La porta del backend è pubblicata per comodità di sviluppo e collaudo, e in produzione andrebbe rimossa lasciando il reverse proxy come unico punto di ingresso.

5.6 Persistenza dei dati

Il filesystem di un container è effimero: quando il container viene rimosso, anche i suoi dati vengono persi. La persistenza è affidata a tre volumi gestiti dall'orchestratore, indipendenti dai container:

- `pgdata` per il database,
- `garage_meta` per gli indici dello storage,
- `garage_data` per le immagini.

Rispetto ai montaggi di directory dell'host, i volumi non dipendono dai suoi percorsi, evitano problemi di permessi e garantiscono prestazioni migliori su piattaforme diverse da Linux.

I montaggi diretti sono usati solo in due casi, entrambi in sola lettura: lo script di inizializzazione del database e la configurazione dello storage. PostgreSQL esegue lo script *solo al primo avvio*, quando il volume è vuoto; modifiche successive non hanno effetto. Per questo contiene soltanto l'abilitazione dell'estensione vettoriale, mentre lo schema viene creato dall'applicazione tramite la procedura idempotente descritta in §4.2.4, così da poter evolvere insieme al codice.

5.7 File per l'ambiente di sviluppo

Un file di sovrapposizione separato aggiunge alla configurazione di produzione ciò che serve durante lo sviluppo: il montaggio del codice sorgente nel container e il ricaricamento automatico del server a ogni modifica.

La scelta del nome è deliberata: usare un nome esplicito impedisce che l'orchestratore attivi automaticamente la configurazione di sviluppo in produzione. Per abilitarla è necessario indicare esplicitamente entrambi i file sulla riga di comando.

Il montaggio riguarda inoltre solo la directory del codice applicativo. Montare l'intero progetto sovrascriverebbe infatti librerie e modelli installati nell'immagine, impedendo al container di avviarsi.

5.8 Configurazione e gestione dei segreti

La configurazione segue il principio della *Twelve-Factor App*: nessun valore specifico dell'ambiente è inserito nel codice e la stessa immagine può essere eseguita in contesti diversi modificando le sole variabili d'ambiente. Nel backend queste sono gestite da una classe di impostazioni validata all'avvio, che segnala subito eventuali variabili mancanti o di tipo errato, evitando che gli errori emergano solo al primo utilizzo. Un'unica istanza è condivisa dai moduli e i percorsi dei file sono calcolati rispetto al modulo che li utilizza, rendendo il comportamento indipendente dalla directory di avvio del processo.

5.9 Avvio del sistema

L'intero sistema si avvia con due comandi:

```
1 cp .env.example .env           # e valorizzare le variabili obbligatorie
2 docker compose up -d --build
```

Listing 5.2: Configurazione e avvio dello stack completo.

L'orchestratore costruisce le immagini, avvia i servizi nell'ordine imposto dalle dipendenze e attende che ciascuna sonda di prontezza dia esito positivo prima di avviare il servizio successivo. Al termine l'interfaccia è raggiungibile su <http://localhost> e la documentazione interattiva dell'API su <http://localhost:8000/docs>. La verifica dello stato dei servizi e la consultazione dei registri avvengono per nome di servizio:

```
1 docker compose ps              # stato e sonde di prontezza
2 docker compose logs -f backend # registri in tempo reale
3 docker compose up -d --scale backend=3 # replicazione del livello
   applicativo
4 docker compose down            # arresto (i volumi sono preservati)
```

Listing 5.3: Comandi operativi principali.

L'esecuzione della suite di test del backend richiede il solo motore Docker attivo, poiché i servizi di persistenza necessari sono creati e distrutti automaticamente dalla suite stessa.

5.10 Estrazione dell'elaborazione in un servizio autonomo

L'elaborazione OCR è il candidato più naturale all'estrazione in un servizio autonomo per tre motivi. Presenta un profilo di risorse particolare, essendo fortemente dipendente dalla CPU e

con tempi di elaborazione variabili in funzione dell'immagine. Introduce inoltre dipendenze di sistema pesanti, tra cui il motore OCR e il pacchetto linguistico italiano, necessari anche quando l'OCR non viene eseguito. Infine, è una componente facilmente sostituibile con un servizio gestito.

L'architettura di destinazione renderebbe esplicito il modello produttore-consumatore già presente logicamente, sostituendo i compiti differiti del framework con una coda di messaggi:

1. l'endpoint salva l'immagine nello storage, pubblica in coda l'identificatore del documento e la chiave dell'oggetto, quindi risponde immediatamente;
2. uno o più consumatori, scalabili indipendentemente dal backend, prelevano il messaggio, scaricano l'immagine ed eseguono l'OCR;
3. il consumatore aggiorna il documento, portandolo allo stato **elaborato** o **errore**.

Il vantaggio principale non sarebbe quindi la sola scalabilità, ma la **durabilità del lavoro**: un messaggio sopravvive al riavvio del consumatore e può essere riconsegnato, superando il limite descritto in §4.5.5.

5.11 Corrispondenza con i servizi cloud gestiti

Ogni componente è stato selezionato in modo da esporre l'interfaccia standard del corrispettivo gestito.

Componente locale	AWS	Azure	Google Cloud
Backend FastAPI	ECS / Fargate	App Service	Cloud Run
PostgreSQL + pgvector	RDS for PostgreSQL	Azure Database	Cloud SQL
Garage (object storage)	S3	Blob Storage	Cloud Storage
Nginx (proxy e statici)	ALB + CloudFront	Front Door	Cloud Load Balancing
Docker Compose	ECS / EKS	AKS	GKE
Tesseract OCR	Textract	AI Vision	Vision API
(coda dei messaggi)	SQS	Service Bus	Pub/Sub

Tabella 5.3: Corrispondenza fra i componenti locali e i servizi gestiti dei principali fornitori.

Due componenti mostrano come la **portabilità** derivi da scelte progettuali precise. **Garage**, grazie all'API S3, può essere sostituito modificando principalmente endpoint e credenziali. **pgvector**, essendo un'estensione di PostgreSQL, mantiene inoltre compatibilità con i principali servizi gestiti e permette agli embedding di restare nella stessa transazione dei metadati.

L'unica componente senza un equivalente gestito diretto è il **modello di embedding**, eseguito localmente. Una soluzione gestita introdurrebbe costi per invocazione, latenza di rete e una dipendenza esterna nel percorso di ricerca. L'esecuzione locale è quindi più adatta al contesto comunale e garantisce che i dati non lascino l'infrastruttura dell'ente.

6 Interfaccia Utente

6.1 Tecnologia e deployment

L'interfaccia è single-page application realizzata con Angular 19, compilata in file statici e servita da Nginx all'interno di un proprio container. Non esiste rendering lato server: il browser riceve un documento HTML minimale e l'applicazione costruisce l'interfaccia interrogando l'API. Il servizio che la serve è quindi privo di stato quanto il backend.

L'applicazione usa componenti autonomi e gestisce il loro stato tramite *signal*, valori che si aggiornano automaticamente quando cambiano i dati da cui dipendono. La struttura è divisa in tre livelli (presentazione, servizi HTTP e modelli di dati). I modelli TypeScript rispecchiano quelli Pydantic del backend, così eventuali differenze vengono rilevate già in fase di compilazione.

Il deployment segue il modello descritto in §5.3. Oltre a servire i file statici, Nginx svolge due funzioni: gestisce il *fallback* del routing lato client, restituendo il documento principale quando una richiesta non corrisponde a un file, così da evitare errori 404 sugli indirizzi profondi, e inoltra le richieste applicative al backend.

6.2 Comunicazione con il backend

Tutte le chiamate usano indirizzi *relativi* (`/api/documenti`, `/api/ricerca`, `/api/upload`), mentre Nginx inoltra al backend le richieste che iniziano con `/api/`.

In questo modo il browser comunica sempre con una *sola origine*, evitando completamente il problema CORS. Inoltre, l'indirizzo del backend non è incluso nel codice compilato, permettendo di usare la stessa immagine in ambienti diversi.

Due parametri del proxy sono adattati al backend. Il limite del corpo è fissato a 12 MB, superiore ai 10 MB dell'applicazione: è quindi FastAPI a gestire i file troppo grandi con un errore JSON comprensibile al frontend. I timeout di lettura e invio sono invece estesi a cinque minuti per consentire l'elaborazione di importazioni massicce senza falsi errori di gateway.

Accesso diretto allo storage: Le immagini costituiscono l'unica eccezione al percorso appena descritto. Il frontend richiede al backend un URL temporaneo firmato e lo assegna direttamente all'attributo di un elemento immagine: i byte viaggiano dallo storage al browser senza attraversare né il backend né il proxy. La scelta alleggerisce il livello applicativo, ma introduce un caso di guasto specifico che l'interfaccia deve gestire, discusso in §6.3.

6.3 Resilienza dell'interfaccia

Un'interfaccia che comunica con un sistema distribuito deve gestire tre condizioni assenti in un'applicazione locale: la risposta può essere lenta, può non arrivare oppure può riferirsi a un'operazione ancora in corso. L'applicazione gestisce esplicitamente tutti e tre i casi.

Caricamento asincrono e stati intermedi espliciti: Ogni componente che interroga l'API espone un segnale di caricamento e uno di errore accanto ai dati: l'interfaccia mostra un

indicatore durante l'attesa e un messaggio esplicito in caso di errore, evitando schermate vuote. I tre stati sono modellati nel componente e non dedotti dall'assenza di dati, che è ambigua (un elenco vuoto può indicare sia un archivio senza documenti sia una richiesta fallita).

Guasti parziali che non propagano: Il principio di degradazione controllata di §4.5.4 si applica anche al client: ogni chiamata gestisce il proprio errore in modo indipendente. Nella dashboard, il fallimento delle statistiche lascia i contatori a zero senza impedire la visualizzazione degli ultimi documenti; nella scheda di dettaglio, il fallimento dei suggerimenti semantici non impedisce la consultazione. Il guasto di una richiesta, quindi, non compromette l'intera pagina.

Attesa attiva del completamento di un'elaborazione: Il caso più significativo riguarda l'asincronia della pipeline. L'endpoint di caricamento risponde subito con il documento in attesa e delega l'elaborazione a un compito in background. La scheda di dettaglio, aperta subito dopo il salvataggio, interrogava quindi il backend una sola volta, arrivando prima della conclusione dell'elaborazione e mostrando un'attesa indefinita senza testo riconosciuto né parole chiave.

La soluzione consiste in un'interrogazione periodica che termina quando lo stato non è più *in attesa*:

```

1 timer(INTERVALLO_POLLING_MS, INTERVALLO_POLLING_MS)
2   .pipe(
3     // switchMap e non mergeMap: se una risposta tarda piu' dell'intervallo,
4     // la richiesta ancora in volo viene annullata invece di accodarsi.
5     switchMap(() => this.documentoService.getDocumento(id)),
6     // inclusive: true lascia passare anche l'ultimo valore, quello che ha
7     // reso falsa la condizione, cioè il documento già elaborato.
8     takeWhile((doc) => doc.stato_effettivo === 'in_attesa', true),
9     takeUntilDestroyed(this.destroyRef),
10  )
11  .subscribe({ /* ... */ });

```

Listing 6.1: Interrogazione periodica dello stato di elaborazione (documento-dettaglio.component.ts, estratto).

Tre dettagli riguardano direttamente la natura distribuita del problema:

Il primo è la scelta dell'operatore che *annulla* la richiesta precedente anziché accodarla: se il backend risponde più lentamente dell'intervallo di interrogazione (ad esempio durante il riconoscimento ottico), le richieste non si accumulano e non arrivano fuori ordine, evitando regressioni nello stato mostrato.

Il secondo è il **tetto massimo** di novanta secondi: se il compito in background termina senza registrare lo stato di errore, il documento non resta in attesa indefinitamente e la pagina smette di interrogare il backend. È la contropartita lato client del limite dichiarato in §4.5.5.

Il terzo è il ricaricamento dei suggerimenti semantici al completamento, poiché il compito in background ricalcola l'embedding includendo il testo riconosciuto.

Infine, l'interrogazione è condizionata allo stato *effettivo*, non a quello *grezzo*. Quest'ultimo vale *in_attesa* anche per una scheda inserita con i soli metadati, per la quale nessun compito è stato avviato. In questo caso il polling segnalerebbe un'elaborazione inesistente. La colonna generata di §4.2.3 evita di replicare questa distinzione in TypeScript.

Avanzamento reale del caricamento massivo: La barra mostra la percentuale di file già trasmessa, calcolata dagli eventi di avanzamento del client HTTP. Per ottenere questi eventi è necessario usare `XMLHttpRequest`: l'API `fetch`, usata come alternativa da Angular, non comunica l'avanzamento dei dati inviati al server, ma solo quello della risposta. Con `fetch`, quindi, la barra resterebbe a zero fino alla fine del caricamento. Si è mantenuta `XMLHttpRequest`, poiché il progetto non utilizza il rendering lato server, unico caso in cui `fetch` sarebbe necessaria.

Immagini non disponibili: L'endpoint che genera l'URL firmato usa solo il riferimento memorizzato e non verifica che l'immagine esista nello storage. Per questo può restituire un esito positivo anche se l'oggetto è stato rimosso, mentre il 404 viene rilevato solo quando il browser apre l'URL. L'interfaccia gestisce il caso mostrando un segnaposto diverso a seconda che la scheda non abbia mai avuto una scansione oppure che questa non sia più disponibile. La distinzione è importante: nel secondo caso l'operatore deve essere avvisato della perdita del documento.

Validazione anticipata lato client: Tipo e dimensione dei file sono verificati nel browser prima dell'invio, usando gli stessi limiti del backend. Questa duplicazione è deliberata: la validazione lato client evita di trasmettere file fino a 12 MB che verrebbero poi rifiutati, mentre quella lato server resta l'unica autorevole, poiché la prima può essere facilmente aggirata tramite chiamate dirette all'API.

6.4 Le viste principali

L'applicazione è organizzata in sei viste raggiungibili da una barra di navigazione laterale, corrispondenti ai percorsi definiti nella configurazione del routing. Le didascalie che seguono descrivono, oltre a ciò che si vede, che cosa accade nel sistema durante la loro visualizzazione.

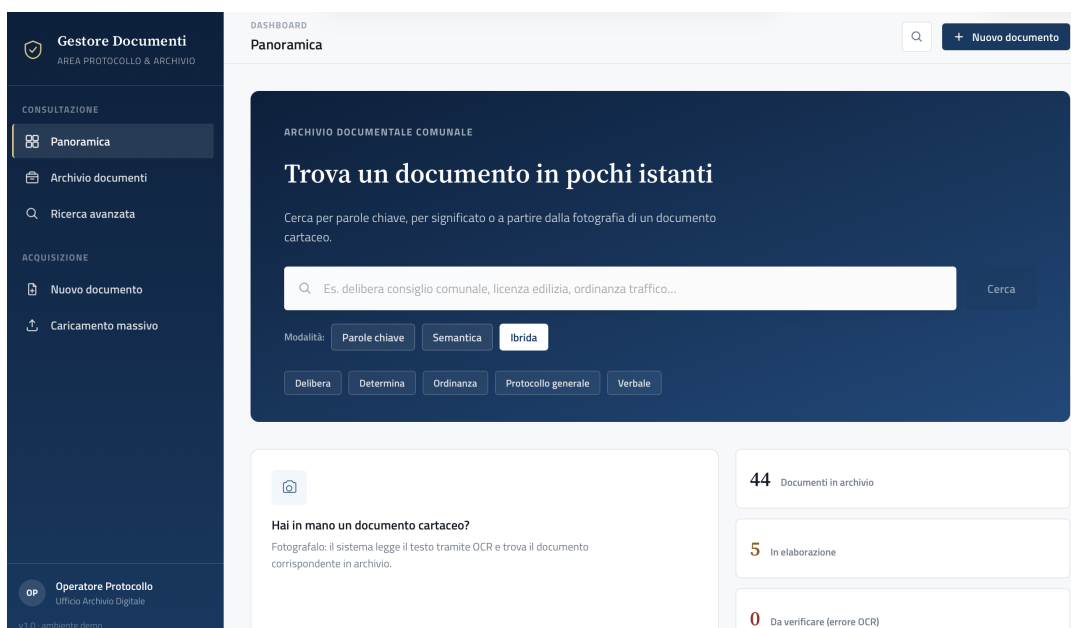


Figura 6.1: Dashboard. La pagina emette due richieste indipendenti e concorrenti: l’endpoint di aggregazione, che calcola i contatori con un’unica interrogazione SQL sull’intero archivio senza trasferire alcuna riga, e l’elenco paginato dei documenti recenti. Le due sono deliberatamente separate perché derivare i contatori dall’elenco produrrebbe numeri errati, fermi al limite di paginazione (§4.6.3). Il fallimento della prima lascia i contatori a zero senza impedire la visualizzazione della seconda.

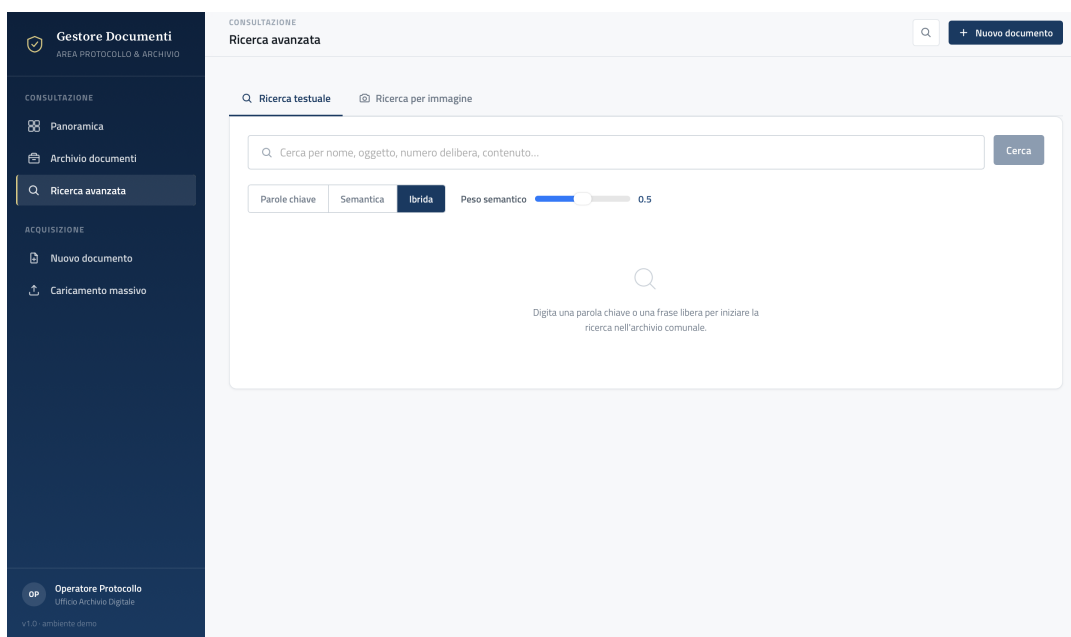


Figura 6.2: Ricerca avanzata. Il selettore di modalità determina quale dei tre endpoint venga interrogato; il cursore del peso semantico agisce sul coefficiente della combinazione ibrida descritta in §4.4.3, ed è quindi esposto solo per quella modalità. Il punteggio mostrato accanto a ciascun risultato è quello calcolato dal backend: per la ricerca full-text la rilevanza normalizzata, per quella semantica la similarità coseno. Il filtro per tipologia opera invece sui risultati già ricevuti, senza generare una nuova richiesta.

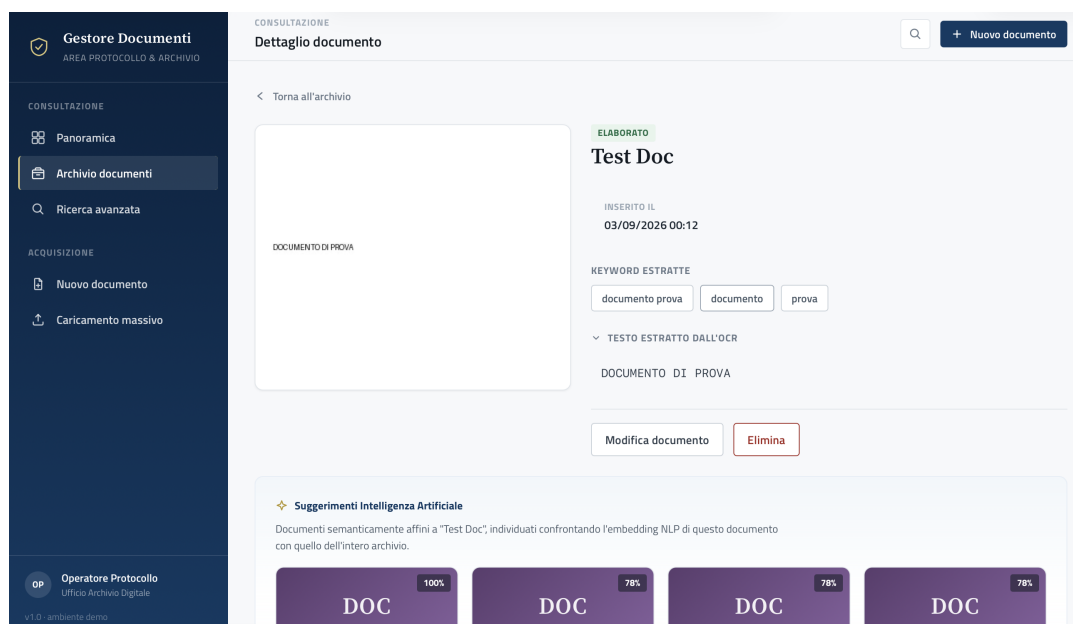


Figura 6.3: Scheda di dettaglio. È la vista che espone il maggior numero di meccanismi distribuiti. L'immagine non transita dal backend ma è prelevata direttamente dallo storage tramite URL firmato; se l'oggetto non è più disponibile, la vista ricade su un segnaposto dedicato. Se il documento è in elaborazione, la pagina avvia l'interrogazione periodica descritta in §6.3 e aggiorna badge, testo riconosciuto e parole chiave non appena il compito in background termina, ricaricando contestualmente i suggerimenti semantici, il cui calcolo dipende dall'embedding appena ricostruito.

Le restanti due viste operative non sono documentate con una schermata dedicata, ma meritano una breve nota sul loro funzionamento.

Nella **ricerca per immagine**, la fotografia viene inviata al backend, che ne estrae il testo e lo usa per una ricerca ibrida (§4.4.4). L'immagine non viene conservata né associata ad alcuna scheda. Il testo riconosciuto viene mostrato all'operatore, così da distinguere un archivio privo del documento cercato da un riconoscimento ottico errato.

Nel **caricamento massivo**, la barra mostra l'avanzamento reale dei byte trasmessi e il riepilogo finale riflette l'isolamento dei guasti parziali del backend (§4.5.4): le righe non valide non interrompono il caricamento e sono riportate con il numero di riga originale, facilitandone l'individuazione nel file.

Completano l'applicazione la **vista di archivio**, con visualizzazione a griglia o tabella, e il **modulo di inserimento e modifica**. Quest'ultimo mostra il suggerimento della tipologia (§4.4.5) come proposta non vincolante e, in modifica, permette di correggere manualmente il testo riconosciuto e le parole chiave, lasciando all'operatore la possibilità di correggere eventuali errori dell'elaborazione automatica.

7 Prompt e Generazioni IA

Lo sviluppo del sistema si è avvalso di assistenti di IA generativa, impiegati come supporto alla scrittura del codice e alla valutazione delle alternative progettuali. Il capitolo raccoglie i prompt formulati e le attività che ne sono derivate, riportando il risultato ottenuto dopo revisione e verifica.

7.1 Prompt formulati e output ottenuti

1. *Query per ottenere l'aggiornamento automatico di `search_vector` a ogni insert e update (in `crea_trigger_search_vector(conn)`).*

Un trigger BEFORE INSERT/UPDATE, con pesi A-D per dare più rilevanza ai campi importanti.

File: backend/app/database.py

```
1 CREATE OR REPLACE FUNCTION documenti_search_vector_update() RETURNS trigger
  AS $$
2 BEGIN
3     NEW.search_vector :=
4         setweight(to_tsvector('italian', coalesce(NEW.nome, '')), 'A') ||
5         setweight(to_tsvector('italian', coalesce(NEW.descrizione, '')),
6             'B') ||
7         setweight(to_tsvector('italian', coalesce(NEW.tipologia, '')), 'C')
8         ||
9         setweight(to_tsvector('italian', coalesce(NEW.testo_ocr, '')), 'D');
10    RETURN NEW;
11 END
12 $$ LANGUAGE plpgsql;
13
14 DROP TRIGGER IF EXISTS trg_documenti_search_vector ON documenti;
15
16 CREATE TRIGGER trg_documenti_search_vector
17     BEFORE INSERT OR UPDATE ON documenti
18     FOR EACH ROW
19     EXECUTE FUNCTION documenti_search_vector_update();
```

2. *In `database.py` come gestisco i lock con FastAPI? Perché se avvio più repliche del backend, rischio che provino tutte insieme a creare lo schema.*

Con un advisory lock di PostgreSQL: è condiviso tra tutte le repliche, mentre un `threading.Lock` proteggerebbe solo il singolo processo.

File: backend/app/database.py

```
1 # Chiave fissa del lock consultivo, dedicata alla sincronizzazione di
  init_db().
2 _SCHEMA_INIT_LOCK_KEY = 72197
3
4 # 1. ACQUISIZIONE DEL LOCK:
5 conn.execute(text("SELECT pg_advisory_lock(:key)"), {"key":
6     _SCHEMA_INIT_LOCK_KEY})
7 try:
8     # 2. SEZIONE CRITICA:
9     # Crea le tabelle definite dai modelli ORM, se mancanti
10    Base.metadata.create_all(bind=conn)
```



```

10     # Installa o aggiorna il trigger della ricerca full-text
11     crea_trigger_search_vector(conn)
12     # Conferma le modifiche allo schema
13     conn.commit()
14 finally:
15     # 3. RILASCIO DEL LOCK E PULIZIA:
16     # Ripristina la transazione prima di rilasciare il lock: dopo un errore,
17     # PostgreSQL non accetta altri comandi finche' non viene eseguito il
18     # rollback.
19     conn.rollback()
20     # Rilascia il lock e permette alle altre repliche di procedere
21     conn.execute(
22         text("SELECT pg_advisory_unlock(:key)"),
23         {"key": _SCHEMA_INIT_LOCK_KEY}
24     )
25     conn.commit()

```

3. In `documento.py` come dichiaro l'indice HNSW per la ricerca semantica sui vettori e l'indice GIN per velocizzare la ricerca full-text sul testo OCR?

File: `backend/app/models/documento.py`

```

1  __table_args__ = (
2      # INDICE AI (HNSW): Organizza i vettori matematici per permettere
3      # la ricerca per similarita' (ricerca semantica) in millisecondi
4      Index(
5          'ix_documenti_embedding_hnsw',
6          'embedding',
7          postgresql_using='hnsw',
8          postgresql_with={'m': 16, 'ef_construction': 64},
9          postgresql_ops={'embedding': 'vector_cosine_ops'}
10     ),
11     # INDICE FULL-TEXT (GIN): Senza questo, cercare una parola nei testi OCR
12     # costringerebbe il DB a leggere la tabella riga per riga
13     Index(
14         'ix_documenti_search_vector_gin',
15         'search_vector',
16         postgresql_using='gin'
17     ),
18     ...
19 )

```

4. Come faccio a far eseguire agli Schemas (DTO) la conversione diretta dagli oggetti ORM?

File: `backend/app/schemas/documento_schemas.py`

```

1  # Permette a Pydantic di creare la risposta direttamente dagli oggetti ORM
2  model_config = ConfigDict(from_attributes=True)

```

5. Calcolo della similarità coseno e normalizzazione dei vettori.

Con vettori a norma 1 la similarità coseno coincide con il prodotto scalare, quindi non serve calcolare le norme.

File: `backend/app/services/embedding_service.py`

```

1      # La normalizzazione rende i vettori confrontabili tramite distanza
      coseno
2      # e mantiene coerenza con l'indice e gli operatori pgvector utilizzati.
3      vettore = model.encode(
4          testo,
5          normalize_embeddings=True
6      )

```

File: backend/app/services/classificazione_service.py

```

1      # Per vettori normalizzati, la similarita' coseno coincide con il
      prodotto
2      # scalare, evitando il calcolo esplicito delle norme dei vettori
3      return float(np.dot(va, vb))

```

6. Nel caricamento massivo sto chiamando la funzione del singolo embedding dentro un ciclo. Come posso migliorare? E se una riga non ha testo, come la gestisco?

Una sola `model.encode()` sui soli testi non vuoti, poi si rimettono i vettori al posto giusto; le righe scartate restano `None`.

File: backend/app/services/embedding_service.py

```

1  def genera_embeddings_batch(testi: list[str]) -> list[list[float] | None]:
2      # Crea una lista dei risultati vuota, della stessa lunghezza della lista
      originale
3      risultati: list[list[float] | None] = [None] * len(testi)
4
5      # Trova gli indici dei soli testi che contengono informazioni
      significative
6      indici_validi = [
7          i for (i, t) in enumerate(testi)
8          if t and t.strip()
9      ]
10
11     if not indici_validi:
12         return risultati
13
14     # Codifica-embedding dei soli testi validi a blocchi di 64
15     # (equilibrio tra velocita' e memoria)
16     vettori = model.encode(
17         [testi[i] for i in indici_validi],
18         normalize_embeddings=True,
19         batch_size=64,
20     )
21
22     # Ripristina ogni embedding nella posizione corrispondente al testo
      originale
23     for posizione, indice in enumerate(indici_validi):
24         risultati[indice] = vettori[posizione].tolist()
25
26     return risultati

```

7. Scikit-learn (che KeyBERT usa dentro per contare gli n-grammi) come stringa accetta solo "english" per le stop word: se gli passo "italian" il `CountVectorizer` mi dà `InvalidParameterError`. Come faccio a usare comunque una lista di stop word italiane?

Gli si passa una **lista** invece della stringa, prendendo le stop word italiane da spaCy.

File: backend/app/services/keyword_service.py

```

1 from spacy.lang.it.stop_words import STOP_WORDS as ITALIAN_STOP_WORDS
2
3 # KeyBERT utilizza scikit-learn, che non fornisce direttamente una lista
4 # italiana. Le stop word di spaCy garantiscono quindi il filtraggio delle
5 # parole comuni senza introdurre una lista manuale.
6 _ITALIAN_STOP_WORDS = list(ITALIAN_STOP_WORDS)

```

```

1 # Esclude le STOP WORD della lingua italiana
2 stop_words=_ITALIAN_STOP_WORDS,

```

8. Per la classificazione zero-shot in `classificazione_service.py` mi servono i Prototipi di Tipologia, cioè testi rappresentativi per ogni tipologia su cui calcolare la similarità. Me li fornisci?

File: backend/app/services/classificazione_service.py

```

1 PROTOTIPI_TIPOLOGIA: dict[str, str] = {
2     "Delibera": (
3         "delibera di giunta comunale approvazione provvedimento deliberativo
4         "
5         "deliberazione consiglio comunale atto deliberativo"
6     ),
7     "Ordinanza": (
8         "ordinanza del sindaco provvedimento contingibile urgente "
9         "ordinanza sindacale limitazione divieto"
10    ),
11    "Determina": (
12        "determina dirigenziale determinazione a contrarre "
13        "affidamento impegno di spesa determina"
14    ),
15    ... # Autorizzazione, Regolamento, Piano, Verbale, Circolare
16 }

```

9. Lock per inizializzare la cache `_prototipi_embedding` in `classificazione_service.py`, per evitare che più richieste insieme calcolino gli stessi embedding.

Double-checked locking: gli embedding dei prototipi si calcolano una volta sola, e a cache già pronta il lock non viene nemmeno acquisito.

File: backend/app/services/classificazione_service.py

```

1 global _prototipi_embedding
2
3 # Evita l'acquisizione del lock dopo l'inizializzazione della cache
4 if _prototipi_embedding is None:
5
6     # Garantisce che un solo thread esegua il caricamento iniziale
7     with _prototipi_lock:
8
9         # Ricontrolla la cache perche' un altro thread potrebbe averla
10        # inizializzata durante l'attesa del lock.
11        if _prototipi_embedding is None:
12            risultati = {}
13

```

```

14         for (nome, testo) in PROTOTIPI_TIPOLOGIA.items():
15             emb = genera_embedding(testo)
16
17             if emb is not None:
18                 risultati[nome] = emb
19
20         _prototipi_embedding = risultati
21
22     return _prototipi_embedding

```

10. *Tesseract apre un thread per core su ogni immagine, e qui ogni upload lancia un OCR in background: con pochi utenti insieme i thread si moltiplicano e va in thrashing. Quali soluzioni possono essere adottate?*

Con `_limita_thread_tesseract()`, chiamata a runtime subito prima dell'OCR: nel Dockerfile la variabile la leggerebbe anche PyTorch.

File: `backend/app/services/ocr_service.py`

```

1 def _limita_thread_tesseract() -> None:
2     # Evita l'oversubscription della CPU quando piu' richieste OCR vengono
3     # elaborate contemporaneamente, riducendo il rischio di CPU thrashing.
4     #
5     # La variabile viene impostata immediatamente prima dell'OCR (e non nel
6     # Dockerfile) per limitare solo Tesseract senza influenzare globalmente
7     # le librerie AI.
8     os.environ.setdefault("OMP_THREAD_LIMIT", "1")

```

```

1 def estrai_testo(self, image_bytes: bytes) -> str:
2     # Applica il limite prima dell'avvio di Tesseract
3     _limita_thread_tesseract()

```

11. *Che pre-processing conviene fare sulle immagini prima di darle a Tesseract?*

File: `backend/app/services/ocr_service.py`

```

1 def _preprocess(self, image: Image.Image) -> Image.Image:
2     # 1. Conversione in scala di grigi
3     # (Tesseract lavora meglio senza rumore cromatico)
4     image = ImageOps.grayscale(image)
5
6     # 2. Aumento del contrasto del 50% (1.5),
7     # utile su scansioni sbiadite o fotocopie
8     enhancer = ImageEnhance.Contrast(image)
9     image = enhancer.enhance(1.5)
10
11     return image

```

12. *Per l'OCR posso leggere l'immagine direttamente dalla memoria, senza scrivere file temporanei sul disco?*

File: `backend/app/services/ocr_service.py`

```

1     # BytesIO consente a Pillow di leggere i dati dalla memoria,
2     # evitando la creazione di file temporanei sul filesystem.
3     image = Image.open(io.BytesIO(image_bytes))

```

13. *Come configuro boto3 in `garage_service.py` per parlare con Garage?**File:* backend/app/services/garage_service.py

```

1      _boto_config = Config(
2          signature_version="s3v4",
3          s3={"addressing_style": "path"},
4          connect_timeout=5,
5          read_timeout=20,
6          retries={"max_attempts": 3, "mode": "standard"},
7      )
8
9      self._client = boto3.client(
10         "s3",
11         endpoint_url=settings.S3_ENDPOINT,
12         aws_access_key_id=settings.S3_ACCESS_KEY,
13         aws_secret_access_key=settings.S3_SECRET_KEY,
14         config=_boto_config,
15         region_name="garage", # valore arbitrario, richiesto da boto3
16     )

```

14. *I presigned URL si portano dentro la firma l'indirizzo del server, se lo genero con l'indirizzo interno di Docker (`http://garage:3900`) ma il browser lo apre da fuori (`http://localhost:3900`), la firma non torna. Come li apro dal browser? Mi serve anche per gli URL temporanei che mostra il frontend.*

Due client boto3: uno interno per le operazioni server-to-server, uno pubblico usato solo per firmare i presigned URL.

File: backend/app/services/garage_service.py

```

1      # Client Pubblico: dedicato agli URL accessibili dal browser.
2      # Signature V4 include l'hostname nella firma, quindi l'endpoint
3      # per generare il link deve coincidere con quello raggiunto dal
4      # client.
5      public_endpoint = (
6          settings.S3_PUBLIC_ENDPOINT
7          or settings.S3_ENDPOINT
8      )
9
10     self._public_client = boto3.client(
11         "s3",
12         endpoint_url=public_endpoint,
13         aws_access_key_id=settings.S3_ACCESS_KEY,
14         aws_secret_access_key=settings.S3_SECRET_KEY,
15         config=_boto_config,
16         region_name="garage",
17     )

```

```

1      def get_presigned_url(
2          self,
3          object_key: str,
4          expires_in: int = 3600
5      ) -> str:
6          # Il client pubblico genera una firma coerente con l'hostname
7          # utilizzato
8          # dal browser, evitando errori di validazione Signature V4.
9          return self._public_client.generate_presigned_url(

```

```

9         "get_object",
10        Params={
11            "Bucket": self._bucket_name,
12            "Key": object_key
13        },
14        ExpiresIn=expires_in,
15    )

```

15. In `create_documento` ho un problema con `embedding_precalcolato`. Se il default è `None` non riesco a distinguere due casi: la creazione singola dal sito, dove l'embedding lo devo calcolare io, e l'import massivo da CSV, dove l'embedding è già stato calcolato in batch ed è `None` solo perché il documento non aveva testo. Così nel secondo caso lo ricalcolo per niente. Che soluzione posso adottare per distinguerli?

Con un oggetto sentinella come valore di default.

File: `backend/app/services/documento_service.py`

```

1  # Distingue un argomento non fornito da un None legittimo (dal caricamento
2  # massivo), che puo' essere un risultato valido per un testo privo di
3  # contenuto.
4
5  _EMBEDDING_NON_FORNITO = object()
6
7  def create_documento(
8      db: Session,
9      documento: DocumentoCreate,
10     embedding_precalcolato=_EMBEDDING_NON_FORNITO
11 ):
12     # Converte il DTO validato in una Entity da salvare
13     db_documento = Documento(**documento.model_dump())
14
15     if embedding_precalcolato is _EMBEDDING_NON_FORNITO:
16         # Genera l'embedding per le operazioni di creazione singola
17         _aggiorna_embedding(db_documento)
18     else:
19         # Riutilizza il risultato batch, anche se esplicitamente None
20         db_documento.embedding = embedding_precalcolato

```

16. Se eseguo la ricerca testuale sui documenti, utilizzando `LIKE '%parola%'`, cercando “commercio” non trovo “commerciale” o “commercianti” e non posso ordinare i risultati per pertinenza, perché il confronto dà solo vero o falso. Inoltre cercando “traffico commercio” vorrei ottenere l'OR tra i termini.

`tsvector/tsquery` con `ts_rank` per il ranking; i termini si uniscono con `||` (OR).

File: `backend/app/services/ricerca_service.py`

```

1  # Costruisco la Text Search Query (tsquery) a partire dal primo termine
2  tsquery = func.plainto_tsquery('italian', termini[0])
3
4  # Unisco ogni termine successivo con l'operatore OR (||), trasformandolo
5  # separatamente per preservare stemming e normalizzazione.
6  for termine in termini[1:]:
7      tsquery = tsquery.op('||')(
8          func.plainto_tsquery('italian', termine)
9      )

```

```

10
11     # ts_rank calcola quanto il documento e' pertinente alla ricerca,
12     # confrontando
13     # il tsvector del documento (search_vector) con il tsquery della ricerca
14     rank = func.ts_rank(
15         Documento.search_vector,
16         tsquery
17     ).label('score')
18
19     # Costruiamo la query
20     statement = (
21         # Seleziona documento e punteggio
22         select(Documento, rank)
23         # Filtra sui risultati che soddisfano il match full-text (@@)
24         .where(Documento.search_vector.op('@@')(tsquery))
25         # Ordina per rilevanza decrescente
26         .order_by(rank.desc())
27         .limit(limit)
28     )

```

17. Nell'import massivo come valido le singole righe?

Con lo stesso schema Pydantic delle API REST, convertendo gli errori di Pydantic in messaggi leggibili per il report.

File: backend/app/utils/csv_parser.py

```

1     try:
2         documento = DocumentoCreate(**riga_pulita)
3         return (documento, None)
4
5     except ValidationError as e:
6         # Convertiamo gli errori strutturati di Pydantic in un messaggio piu'
7         # leggibile per il report di importazione.
8         messaggi = []
9
10        for err in e.errors():
11            posizione = ".".join(str(parte) for parte in err["loc"])
12            messaggi.append(f"{posizione}: {err['msg']}")
13
14        return (
15            None,
16            RigaErrore(
17                riga=numero_riga,
18                errore="; ".join(messaggi),
19                dati=riga_pulita,
20            ),
21        )

```

18. Il file CSV che carica l'utente arriva dall'API come bytes e io lo decodifico in una stringa, tutto in memoria. Però `csv.DictReader` vuole un oggetto tipo file da leggere riga per riga, non una stringa. Come glielo do senza salvarlo prima su disco?

File: backend/app/utils/csv_parser.py

```

1     # utf-8-sig gestisce anche il BOM (Byte Order Mark)
2     testo = contenuto.decode("utf-8-sig")

```

```

1      # StringIO permette a csv.DictReader di lavorare sui dati già presenti
      in
2      # memoria, senza creare un file temporaneo sul disco.
3      reader = csv.DictReader(io.StringIO(testo))

```

19. Come controllo che nel CSV almeno una colonna corrisponda ai campi che mi aspetto?

Con l'intersezione tra header e whitelist: senza questo controllo ogni riga verrebbe svuotata e l'import finirebbe a zero risultati senza segnalare nessun errore.

File: backend/app/utils/csv_parser.py

```

1  CAMPI_ATTESI = set(DocumentoCreate.model_fields.keys())

2
3
4  # Se nessuna colonna dell'header corrisponde ai campi attesi, ogni riga
5  # verrebbe svuotata da _pulisci_riga e scartata come "vuota", producendo
6  # un import a zero risultati senza alcun errore.
7  if not (set(reader.fieldnames) & CAMPI_ATTESI):
8      risultato.errori.append(
9          RigaErrore(
10             0,
11             f"Nessuna colonna riconosciuta nell'header. "
12             f"Campi attesi: {'', '}.join(sorted(CAMPI_ATTESI))}"
13         )
14     )
15     return risultato

```

20. Per il JSON come faccio lo stesso controllo fatto nel CSV? Perché non c'è un header unico e ogni oggetto ha le sue chiavi.

Il controllo va fatto per singolo elemento: se dopo la pulizia non resta nessun campo ma l'oggetto non era vuoto, si segnala l'errore.

File: backend/app/utils/csv_parser.py

```

1      riga_pulita = _pulisci_riga(elemento)
2
3      # A differenza del CSV, ogni oggetto JSON ha chiavi proprie.
4      # Se contiene solo campi non riconosciuti, verrebbe
5      # scartato silenziosamente: lo intercettiamo.
6      if not riga_pulita and elemento:
7          risultato.errori.append(
8              RigaErrore(
9                  indice,
10                 f"Nessun campo riconosciuto. "
11                 f"Campi attesi: {'', '}.join(sorted(CAMPI_ATTESI))",
12                 dati=elemento,
13             )
14         )
15         continue

```

21. In routers/upload.py come faccio a lanciare l'OCR in background, così l'utente non resta ad aspettare? (_processa_ocr_in_background)

Con `BackgroundTasks` di FastAPI, e dentro una sessione DB **nuova**: quella della richiesta viene chiusa insieme alla risposta.

File: `backend/app/routers/upload.py`

```

1      # L'utente riceve subito la risposta HTTP mentre OCR, keyword extraction
2      # e aggiornamento dell'embedding vengono eseguiti successivamente.
3      background_tasks.add_task(
4          _processa_ocr_in_background,
5          documento_id,
6          file_bytes
7      )

```

```

1  def _processa_ocr_in_background(
2      documento_id: int,
3      image_bytes: bytes
4  ):
5      db = SessionLocal()
6
7      try:
8          ...
9          # ESTRAZIONE DEL TESTO
10         testo = ocr_service.estrai_testo(image_bytes)
11         documento.testo_ocr = testo if testo else None
12
13         # ESTRAZIONE DELLE KEYWORD
14         # Le keyword vengono generate solo se l'OCR ha prodotto testo utile.
15         if testo:
16             kw = estrai_keywords(testo)
17             documento.keywords = kw if kw else None
18
19         # AGGIORNAMENTO DELLO STATO
20         documento.stato_elaborazione = "elaborato" if testo else "errore"
21
22         # AGGIORNAMENTO DELL'EMBEDDING
23         _aggiorna_embedding(documento)
24
25         db.commit()
26         ...
27     except Exception as e:
28         logger.error("Errore OCR per il documento %d: %s", documento_id, e)
29         db.rollback()
30         ...
31     finally:
32         # Chiudere sempre la sessione privata della background task
33         db.close()

```

22. Il CSS di base e parti ripetitive, `styles.scss`

Quattro blocchi: design token in `:root`, reset e accessibilità, scrollbar, primitive riutilizzabili (`.btn`, `.field`, `.textarea`, `.card`, `.badge`, `.chip`, `.visually-hidden`).

File: `frontend/src/styles.scss`

```

1  :root {
2      --color-primary-600: #175c98;
3      ...
4      --font-sans: "Titillium Web", "Segoe UI", system-ui, -apple-system,
5          sans-serif;
6      ...

```

```

6  --space-4: 1rem;
7  ...
8  --radius-md: 6px;
9  ...
10 --sidebar-width: 264px;
11 ...
12 --dur-base: 180ms;
13 }

1  /* Mantiene sempre visibile il focus da tastiera */
2  :focus-visible {
3    outline: 2px solid var(--color-primary-600);
4    outline-offset: 2px;
5    border-radius: var(--radius-xs);
6  }

```

23. Nel frontend come faccio a tenere il menù sempre fisso mentre cambio pagina?

Il menu va messo in `app.component.html`, usando il `router-outlet` per far cambiare solo la parte centrale.

File: `frontend/src/app/app.component.html`

```

1  <app-navbar [mobileOpen]="mobileNavOpen()" (closeMobile)="closeMobileNav()"
2    />
3
4  <div class="shell">
5    <header class="topbar">
6      ...
7    </header>
8
9    <main class="content">
10     <router-outlet />
11   </main>
12 </div>

```

24. Nel componente della libreria come faccio il caricamento incrementale dei documenti, tipo il pulsante “carica altri”?

Si passa come `skip` la lunghezza della lista già caricata e si accodano i nuovi risultati.

File: `frontend/src/app/components/documento-lista/documento-lista.component.ts`

```

1  caricaAltri(): void {
2    this.caricamentoAltri.set(true);
3    this.caricaPagina(this.documenti().length);
4  }
5
6  private caricaPagina(skip: number): void {
7    this.documentoService.getDocumenti(skip, DIMENSIONE_PAGINA).subscribe({
8      next: (pagina) => {
9        // La prima pagina sostituisce l'archivio; le successive vengono
10         accodate
11        // per realizzare il caricamento incrementale senza perdere i dati
12         visibili
13        this.documenti.set(skip === 0 ? pagina : [...this.documenti(),
14          ...pagina]);
15        this.altriDisponibili.set(pagina.length === DIMENSIONE_PAGINA);

```

```

13     this.caricamento.set(false);
14     this.caricamentoAltri.set(false);
15   },
16   ...
17 });
18 }

```

25. Vorrei dare a ogni tipologia di documento un tono di colore diverso, ma senza salvarmi il colore nel documento e facendo in modo che resti sempre lo stesso in tutte le viste. Come faccio?

Si calcola un hash della stringa tipologia e lo si usa come indice su una palette fissa.

File: frontend/src/app/components/documento-card/documento-card.component.ts

```

1  const PALETTE_TIPOLOGIA = ['tono-a', 'tono-b', 'tono-c', 'tono-d', 'tono-e']
   as const;
2
3  /**
4   * Associa ogni tipologia a un tono in modo deterministico, evitando di
   memorizzare
5   * lo stile nel documento e mantenendo la stessa resa tra visualizzazioni
   diverse.
6   */
7  function tonoPerTipologia(tipologia: string | null | undefined): string {
8     const chiave = tipologia?.trim() || 'generico';
9     let hash = 0;
10    for (let i = 0; i < chiave.length; i++) {
11        hash = (hash * 31 + chiave.charCodeAt(i)) >>> 0;
12    }
13    return PALETTE_TIPOLOGIA[hash % PALETTE_TIPOLOGIA.length];
14 }

```

26. Come implemento nel frontend la barra che mostra l'avanzamento del caricamento?

HttpRequest con reportProgress: true nel service, poi nel componente si legge l'evento UploadProgress.

File: frontend/src/app/services/upload.service.ts

```

1  // HttpRequest con reportProgress abilita gli eventi necessari per mostrare
2  // l'avanzamento reale dell'upload massivo
3  caricaMassivo(file: File):
   Observable<HttpEvent<RisultatoCaricamentoMassivo>> {
4     const formData = new FormData();
5     formData.append('file', file);
6
7     const req = new HttpRequest<FormData>(
8         'POST',
9         `${this.baseUrl}/massivo`,
10        formData,
11        {
12            reportProgress: true,
13        }
14    );
15
16    return this.http.request<RisultatoCaricamentoMassivo>(req);
17 }

```

File: frontend/src/app/components/upload-massivo/upload-massivo.component.ts

```
1      // UploadProgress consente di mostrare l'avanzamento reale,  
2      // anziche' simulare una percentuale durante il trasferimento del  
3      // file  
4      if (evento.type === HttpEventType.UploadProgress && evento.total) {  
5          this.percentualeAvanzamento.set(Math.round((evento.loaded /  
6              evento.total) * 100));  
7      } else if (evento.type === HttpEventType.Response && evento.body) {  
8          this.risultato.set(evento.body);  
9          this.caricamentoInCorso.set(false);  
10     }
```

File: frontend/src/app/components/upload-massivo/upload-massivo.component.html

```
1     <div class="progress">  
2         <div class="progress__track">  
3             <div class="progress__fill"  
4                 [style.width.%"="percentualeAvanzamento()"></div>  
5             </div>  
6             <span class="progress__label">Caricamento in corso... {{  
7                 percentualeAvanzamento() }}%</span>  
8         </div>
```